Quaderno AI

Concetti, strumenti e percorsi pratici per lo sviluppo software con AI

Andrea Merlin

Numero 1 - Novembre 2026

Indice

Capitolo 3 **1**

GitHub Spec Kit: sviluppo guidato dalle specifiche con esempi in C# . . . 1

3.1 Lo spec-driven development: invertire la piramide . . . 2

3.2 Cos'è GitHub Spec Kit . . . 4

3.3 Installazione e inizializzazione . . . 7

3.4 La struttura di un progetto Spec Kit . . . 9

3.5 Il workflow a colpo d'occhio . . . 12

3.6 La Constitution: i principi del progetto . . . 15

3.7 `/speckit-specify`: dalla richiesta alla specifica . . . 18

3.8 `/speckit-clarify`: risolvere le ambiguità . . . 22

3.9 `/speckit-plan`: il piano tecnico . . . 24

3.10 `/speckit-checklist`: i gate di qualità della specifica . . . 28

3.11 `/speckit-tasks`: la scomposizione in task . . . 29

3.12 `/speckit-analyze`: la verifica di coerenza . . . 32

3.13 Implement e Converge . . . 34

3.14 Il CLI `specify` . . . 35

3.15 Esempio C#: entità, DbContext e DTO . . . 38

3.16 Esempio C#: gli endpoint della Minimal API . . . 40

3.17 Esempio C#: i test (test-first) . . . 44

3.18 Esempio C#: eseguire, verificare e convergere . . . 48

3.19 Integrazione con Copilot e VS Code . . . 50

3.20 CI/CD e automazione . . . 52

3.21 Best practice e anti-pattern . . . 54

3.22 Il processo di bug fixing . . . 56

3.23 Il processo di idea assessment . . . 57

3.24 Confronti, evoluzione e progetti esistenti . . . 59

3.25 Risoluzione dei problemi comuni . . . 61

3.26 Riepilogo, glossario ed esercizi 63

Capitolo 3

GitHub Spec Kit: sviluppo guidato dalle specifiche con esempi in C#

Abstract. Nei primi due capitoli abbiamo costruito software "a mano": scrivevamo il codice e, al massimo, lo descrivevamo mentre lo producevamo. Esiste però un approccio diverso e sempre più diffuso: **capovolgere la piramide** e partire dalla *specifica* — il *cosa* e il *perché* — per poi derivare piano, task e codice in modo strutturato. **GitHub Spec Kit** è il toolkit open source che rende questo approccio concreto per gli agenti di codifica AI. In questo capitolo capiremo come funziona, ne esploreremo comandi, template e meccanismi, e lo applicheremo per intero a una feature reale in **C# e .NET 10**.

Cosa imparerai in questo capitolo

Al termine della lettura e della pratica saprai:

- spiegare cos'è lo **spec-driven development** e perché inverte il rapporto tradizionale tra specifica e codice;
- installare Spec Kit (CLI `specify` + integrazione con GitHub Copilot o altri agenti);
- orientarti nella struttura di un progetto Spec Kit (`.specify/`, `specs/`, template, file di stato);
- usare il **workflow completo**: constitution, specify, clarify, plan, checklist, tasks, analyze, implement, polish e converge;
- leggere e compilare i **template ufficiali** (spec, plan, tasks, checklist, constitution);
- applicare l'intero flusso a una feature **C#** (una Minimal API ASP.NET Core con EF Core), dalla specifica al codice funzionante;

- integrare Spec Kit con **VS Code e Copilot** e automatizzare i controlli in **CI/CD**;
- riconoscere le **best practice** e gli errori comuni.

Prerequisiti

- .NET SDK 10 e un editor (VS Code con GitHub Copilot, oppure un altro agente supportato);
- Python 3.11+ e `uv` per installare il CLI (vedremo come);
- familiarità con C# e ASP.NET Core (per l'esempio pratico);
- nessuna conoscenza pregressa di Spec Kit: è l'oggetto di questo capitolo.

Una nota sulla versione

Spec Kit è in rapida evoluzione: i nomi dei comandi, i percorsi e i template cambiano tra una versione e l'altra. Questo capitolo usa gli esempi di comando con trattino (`/speckit-specify`, `/speckit-plan`) e descrive la directory `specs/` per le feature e lo stato in `.specify/feature.json`. La sintassi effettivamente invocabile dipende dall'agente e dalle skill generate dall'integrazione: fai riferimento ai comandi presenti nel progetto, senza dedurre dalla punteggiatura che una forma sia necessariamente obsoleta.

3.1 Lo spec-driven development: invertire la piramide

Prima di parlare dello strumento, capiamo la **filosofia** che lo anima. GitHub Spec Kit è l'implementazione pratica dello *Specification-Driven Development* (SDD), un approccio che cambia il ruolo della specifica nel ciclo di vita del software.

3.1.1 Il potere invertito

Per decenni il codice è stato il re. Le specifiche servivano il codice: si scrivevano documenti di requisiti per guidare lo sviluppo, e li si **scartava** appena cominciava il "vero lavoro". Il PRD (Product Requirements Document) diventava rapidamente obsoleto, il design doc finiva in un cassetto, e la fonte di verità rimaneva, di fatto, solo il codice.

Lo spec-driven development inverte questa gerarchia: **la specifica è la fonte di verità**, e il codice è una sua **derivazione**. La specifica non è un artefatto usa-e-getta, ma un documento vivo che viene portato avanti attraverso la pianificazione, l'implementazione e la convergenza finale.

3.1.2 Perché proprio adesso

Tre fattori rendono questo approccio possibile (e conveniente) oggi, quando in passato era impraticabile:

1. **Gli agenti AI possono scrivere molto codice in fretta.** Il collo di bottiglia non è più digitare il codice, ma decidere *cosa* deve fare. Investire tempo nella specifica diventa l'investimento a più alto rendimento.
2. **Gli LLM lavorano bene con strutture.** Un template ben progettato "guida" il modello verso output di qualità costante, riducendo divagazioni e dettagli prematuri.
3. **I team sono distribuiti e asincroni.** Una specifica versionata, rivista e approvata diventa il contratto condiviso tra chi definisce e chi implementa — umano o agente che sia.

3.1.3 L'idea in una frase

> Definisci **cosa** costruire e **perché**, poi trasforma la specifica in un piano, in task e in codice funzionante.

Tutto il resto — comandi, template, checklist — è al servizio di questa frase.

3.1.4 Il flusso SDD in pratica

Il flusso parte da un'idea, spesso vaga e incompleta:

1. **Specifica.** Attraverso un dialogo con l'agente, l'idea diventa una specifica strutturata: user story prioritarie, criteri di accettazione, requisiti funzionali, entità, criteri di successo misurabili.
2. **Chiarimento.** Le ambiguità vengono esplicitate e risolte (o marcate come `[NEEDS CLARIFICATION]`).
3. **Piano.** La specifica diventa un piano tecnico: stack, architettura, modello dati, contratti.
4. **Task.** Il piano viene scomposto in task ordinati per dipendenza, raggruppati per user story.
5. **Implementazione.** I task vengono eseguiti nell'ordine giusto.

6. **Convergenza.** Il codice viene confrontato con la specifica: se manca qualcosa, si aggiungono task e si itera finché non c'è convergenza.

Ciò che richiedeva giorni di riunioni e documentazione si comprime in ore di lavoro focalizzato. E, a differenza del passato, gli artefatti non vengono scartati: restano, versionati, come documentazione viva del sistema.

3.1.5 Il ruolo dei template

La qualità dell'output dipende in modo cruciale dai **template**. Un buon template è, di fatto, un *prompt strutturato* che vincola il modello in modi produttivi:

- **impedisce i dettagli implementativi prematuri** (la specifica non deve dire "usa React", deve dire "l'utente deve vedere aggiornamenti in tempo reale");
- **forza i marcatori di incertezza** (`[NEEDS CLARIFICATION]`) invece di lasciare che il modello inventi;
- **include checklist** che funzionano come "unit test" per la specifica;
- **impone gate di conformità** ai principi del progetto (la *constitution*).

Vedremo ciascuno di questi meccanismi in dettaglio nelle prossime sezioni, perché sono il cuore di Spec Kit.

3.1.6 Cosa NON è lo SDD

Per evitare fraintendimenti:

- non è "scrivere documenti invece di codice": è scrivere **prima** i documenti giusti, per scrivere **meglio** il codice dopo;
- non elimina la revisione umana: la sposta, anzi, più a monte, dove costa meno;
- non è un processo a cascata rigido: le fasi sono iterative (si può tornare indietro e rivedere la specifica);
- non è legato a un linguaggio: funziona con C#, Python, TypeScript, qualunque stack.

3.2 Cos'è GitHub Spec Kit

GitHub Spec Kit è un **toolkit open source** (repository `github/spec-kit`, licenza MIT) che fornisce agli agenti di codifica AI *processi struttu-*

rati, *template riutilizzabili* e *risultati documentati*. Non è un framework applicativo né un linguaggio: è un **livello di metodo** che si appoggia ai coding agent esistenti (GitHub Copilot, Claude Code, Cursor e altri).

3.2.1 I tre processi pronti all'uso

Spec Kit offre tre punti di ingresso indipendenti — non fasi obbligatorie di un unico processo:

Cosa ti serve	Processo	Risultato
Costruire una feature o un'app	**Spec-Driven Development** (incluso nel core)	Una specifica portata attraverso piano, implementazione e convergenza
Diagnosticare e riparare un comportamento rotto	**Bug fixing** (estensione)	Una causa valutata, una correzione mirata e una verifica registrata
Decidere se un'idea merita investimento	**Idea assessment** (estensione)	Una decisione motivata: `go`, `needs-clarification` o `kill`

In questo capitolo ci concentriamo sul processo principale, lo **SDD**; nelle sezioni finali accenniamo agli altri due, che seguono la stessa filosofia.

3.2.2 Le capabilities: un toolkit componibile

Oltre ai processi, Spec Kit espone un insieme di meccanismi componibili:

- **Integration** — collegamento ai diversi coding agent (Copilot, Claude Code, Cursor, ecc.);
- **Extension** — aggiunta di comandi, gate di qualità o processi interi;
- **Preset** — adattamento di template, comandi e terminologia agli standard del team;
- **Workflow** — automazione di prompt, comandi, gate e task di shell;

- **Bundle** — pacchetti riutilizzabili di estensioni, preset e workflow (per ruoli o team);
- **Catalog** — catalogo di componenti della comunità, con la possibilità di ospitarne di privati;
- **Project-local override** — sostituzione di un singolo template in un progetto.

Questa componibilità è ciò che distingue Spec Kit da una semplice raccolta di prompt: puoi partire con il processo pronto, adattarlo al tuo team e persino costruirne uno tuo.

3.2.3 Come si usa, in estrema sintesi

1. Si installa il **CLI** (`specify`) con `uv tool install specify-cli`.
2. Si inizializza un progetto con `specify init <nome> --integration copilot`.
3. Si apre il progetto nell'agente e si invocano i **comandi** (`/speckit-specify`, `/speckit-plan`, …) **nella chat dell'agente**, uno alla volta, rivedendo ogni risultato prima di proseguire.

Il CLI fa il lavoro "di terra" (scaffolding, controlli dei prerequisiti, gestione di estensioni e cataloghi); i comandi esposti dall'integrazione fanno il lavoro "di testa", guidando l'agente nella produzione degli artefatti.

3.2.4 Una breve storia

Spec Kit nasce dall'esperienza di **GitHub Next** e dal progetto precedente noto come *Specify*, poi evoluto e donato come progetto open source dell'organizzazione `github`. I comandi disponibili e la loro sintassi dipendono dall'integrazione e dalle skill che questa genera per l'agente: nelle sezioni seguenti usiamo la forma con trattino mostrata dall'integrazione dell'esempio. Se un progetto espone una forma diversa, va seguita quella effettivamente generata; la punteggiatura da sola non indica che si tratti di una generazione precedente.

3.2.5 Perché dovrebbe interessarti

Se sviluppi con un coding agent, Spec Kit risponde a tre domande ricorrenti:

- "Come faccio a far produrre all'agente **esattamente ciò che voglio**, e non qualcosa di plausibile ma sbagliato?" — con una specifica strutturata e rivista;

- "Come **traccio** il lavoro e lo rendo revisionabile da un team?" — con artefatti versionati in `specs/`;
- "Come impongo gli **standard** del mio team all'agente?" — con la constitution, i preset e le checklist.

È, in sintesi, il modo per passare da "chiedo una cosa alla volta all'agente" a "dirigo un processo di sviluppo documentato e ripetibile".

3.3 Installazione e inizializzazione

Spec Kit ha due componenti da installare: il **CLI** (uno strumento da terminale) e l'**integrazione** con il tuo coding agent (i comandi `/speckit-*`).

3.3.1 I prerequisiti

Ti servono:

- **Python 3.11 o superiore**;
- `uv`, il gestore di pacchetti Python veloce (il CLI `specify` è distribuito come pacchetto Python);
- un coding agent supportato: GitHub Copilot, Claude Code, Cursor, Windsurf, Qoder, Amazon Q, Antigravity, Crush, Gemini CLI, Kilo Code, OpenCode, Roo Code e altri;
- sistema operativo Linux, macOS o Windows.

3.3.2 Installare il CLI

```
uv tool install specify-cli
```

Il comando installa l'eseguibile `specify` nel tuo ambiente. Verifica l'installazione:

```
specify --version
```

> **Nota.** `uv` è il metodo di installazione raccomandato nella documentazione ufficiale. Esistono anche altri installer (release pinnate, pacchetti per CI); per questo capitolo usiamo `uv`, che è il percorso più semplice.

3.3.3 Inizializzare un progetto

```
specify init taskify --integration copilot
cd taskify
```

Il comando:

1. crea la cartella del progetto;
2. genera la struttura di base di Spec Kit (`.specify/` con template, constitution e configurazione);
3. configura l'**integrazione** scelta (`--integration copilot`): per Copilot, crea le skill `/speckit-*` nella cartella `.github/` (o equivalente), pronte per essere invocate nella chat dell'agente;
4. se interattivo, ti chiede alcune preferenze (per esempio la variante degli script di automazione: Bash, PowerShell o Python).

Dopo `init`, apri la cartella nel tuo agente (per esempio `code taskify` in VS Code) e sei pronto a lavorare.

3.3.4 Inizializzare in un progetto esistente

Spec Kit può essere adottato anche in un repository già esistente: la procedura è la stessa, eseguita dentro la cartella del progetto:

```
cd mio-progetto-esistente
specify init . --integration copilot
```

In questo caso Spec Kit aggiunge la sua struttura senza toccare il codice esistente, e puoi iniziare a usare il flusso SDD per le **nuove** feature (e, con le guide appropriate, per far evolvere le specifiche esistenti).

3.3.5 Le opzioni di `init`

`specify init` accetta opzioni utili:

Opzione	Scopo
`--integration <agente>`	Configura l'agente (copilot, claude, cursor, …)
`--script sh|ps|py`	Sceglie la variante degli script di automazione
`--non-interactive`	Esecuzione senza prompt (per CI)

In modalità non interattiva (nessun terminale, o `--non-interactive`), Spec Kit sceglie automaticamente una variante di script adatta al sistema operativo.

3.3.6 Aggiornare

Per aggiornare un'installazione esistente:

```
uv tool upgrade specify-cli
```

L'upgrade del CLI non sovrascrive i tuoi template e le tue specifiche: aggiorna il tool, lasciando intatti gli artefatti del progetto (che restano versionati in Git).

3.3.7 Cosa succede alla prima esecuzione

Alla prima invocazione di un comando `/speckit-*`, l'agente legge la configurazione del progetto (`.specify/`) e i template, e produce il primo artefatto (per esempio `specs/<numero>-<slug>/spec.md`). Non c'è un "server" da avviare: tutto avviene tramite file Markdown versionati e il dialogo con l'agente. Questa semplicità — nessun servizio, nessun database — è parte del fascino dello strumento.

3.4 La struttura di un progetto Spec Kit

Capire la struttura delle cartelle è essenziale per orientarsi e per personalizzare il toolkit. Ecco l'anatomia di un progetto inizializzato.

3.4.1 La struttura d'insieme



Figura 1: Anatomia di un progetto Spec Kit dopo `specify init`

3.4.2 La cartella `.specify/`

È il "cervello" della configurazione. Contiene:

- **`feature.json`** — registra quale feature è **attiva** in questo momento. I comandi `/speckit-*` risolvono la feature da questo stato, **non** dal branch Git corrente. È ciò che permette di lavorare su una feature senza fare `git checkout`, e di cambiarla aggiornando il file (o la variabile d'ambiente `SPECIFY_FEATURE_DIRECTORY`).
- **`init-options.json`** — conserva le scelte fatte in fase di `init`, in particolare la **numerazione** delle feature (`feature_numbering`): da qui deriva il nome della cartella, per esempio `003-user-auth`.
- **`memory/constitution.md`** — i principi stabili del progetto, modificabili solo attraverso le regole di governance. È la "legge" che i comandi di piano e implementazione devono rispettare.
- **`templates/`** — se presenti, i template in questa cartella **sostituiscono** quelli di default per il singolo progetto (project-local override).
- **`extensions.yml`** — (facoltativo) registra estensioni e **hook**, cioè azioni da eseguire prima o dopo i comandi (per esempio creare il

branch Git all'avvio di una feature).

3.4.3 La cartella `specs/`

È dove vivono gli **artefatti di ogni feature**. Ogni feature ha una cartella con un nome del tipo `<prefisso>-<slug>` (per esempio `001-task-manager`), che contiene i file prodotti dai comandi:

Artefatto	Comando e scopo
`spec.md`	`/speckit-specify`: specifica (cosa e perché)
`plan.md`	`/speckit-plan`: piano tecnico (come)
`research.md`	`/speckit-plan`: ricerche tecniche (fase 0)
`data-model.md`	`/speckit-plan`: modello dati (fase 1)
`quickstart.md`	`/speckit-plan`: scenari chiave di validazione
`contracts/`	`/speckit-plan`: contratti di endpoint ed eventi
`tasks.md`	`/speckit-tasks`: task eseguibili
`checklists/requirements.md`	`/speckit-specify` e `/speckit-clarify`: checklist di qualità dei requisiti

3.4.4 Lo stato della feature attiva

Il file `.specify/feature.json` ha una forma simile a:

```
{
  "feature_directory": "specs/001-task-manager"
}
```

Quando invochi `/speckit-plan` o `/speckit-tasks`, il comando legge questo file e sa **dove** andare a lavorare. È un dettaglio piccolo ma potente: svincola il flusso dal branch Git e lo rende deterministico (e più facile da automatizzare in CI).

3.4.5 Il codice sorgente

Spec Kit non impone una struttura al codice: la struttura del codice viene decisa **nel piano** (il file `plan.md` contiene una sezione "Project Structure" con l'albero delle cartelle scelto). Per un'applicazione web, per esempio, il piano potrà proporre `src/` e `tests/`, o `backend/` e `frontend/`. I task faranno riferimento a quei percorsi.

3.4.6 Dove vengono creati i file: un riepilogo

- **`specify init`** crea `.specify/`, i template di default e la constitution;
- **`/speckit-constitution`** aggiorna `.specify/memory/constitution.md`;
- **`/speckit-specify`** crea `specs/<slug>/spec.md` e `checklists/requirements.md`;
- **`/speckit-plan`** crea `plan.md`, `research.md`, `data-model.md`, `quickstart.md` e `contracts/`;
- **`/speckit-tasks`** crea `tasks.md`;
- **`/speckit-implement`** crea il codice sorgente vero e proprio.

Con questa mappa in testa, il resto del capitolo scorrerà molto più facilmente.

3.5 Il workflow a colpo d'occhio

Spec Kit definisce due percorsi per lo SDD: uno **breve** (per feature piccole) e uno **completo** (per feature di produzione). In entrambi i casi, i comandi si invocano **nella chat dell'agente**, uno alla volta, rivedendo ogni risultato prima di procedere.

3.5.1 Il percorso breve

Per feature piccole o prototipi:

1. `/speckit-specify` → la specifica (`spec.md`)
2. `/speckit-plan` → il piano tecnico (`plan.md`, …)
3. `/speckit-tasks` → i task (`tasks.md`)
4. `/speckit-implement` → il codice
5. `/speckit-converge` → verifica della completezza

3.5.2 Il percorso completo

Per feature di produzione, con i gate di qualità:

1. `/speckit-constitution` (una volta per progetto)
2. `/speckit-specify`
3. `/speckit-clarify` → risolve le ambiguità
4. `/speckit-plan`
5. `/speckit-checklist` → checklist di qualità della specifica
6. `/speckit-tasks`
7. `/speckit-analyze` → verifica di coerenza tra gli artefatti
8. `/speckit-implement`
9. **Polish** → revisione mirata, rifinitura e test del risultato, senza ampliare la specifica
10. `/speckit-converge` → verifica della completezza

Polish è un'attività di revisione, non un comando standard di Spec Kit: si esegue dopo l'implementazione e si ripetono i test prima della convergenza. Il diagramma mostra i comandi del flusso e perciò non rappresenta questa attività trasversale. Se l'integrazione offre una skill dedicata, usane la sintassi generata.

Le fasi aggiuntive (`clarify`, `checklist`, `analyze`) sono **gate di qualità**: costano poco, si eseguono in chat e intercettano i problemi prima che diventino codice sbagliato.

3.5.3 Il diagramma del flusso

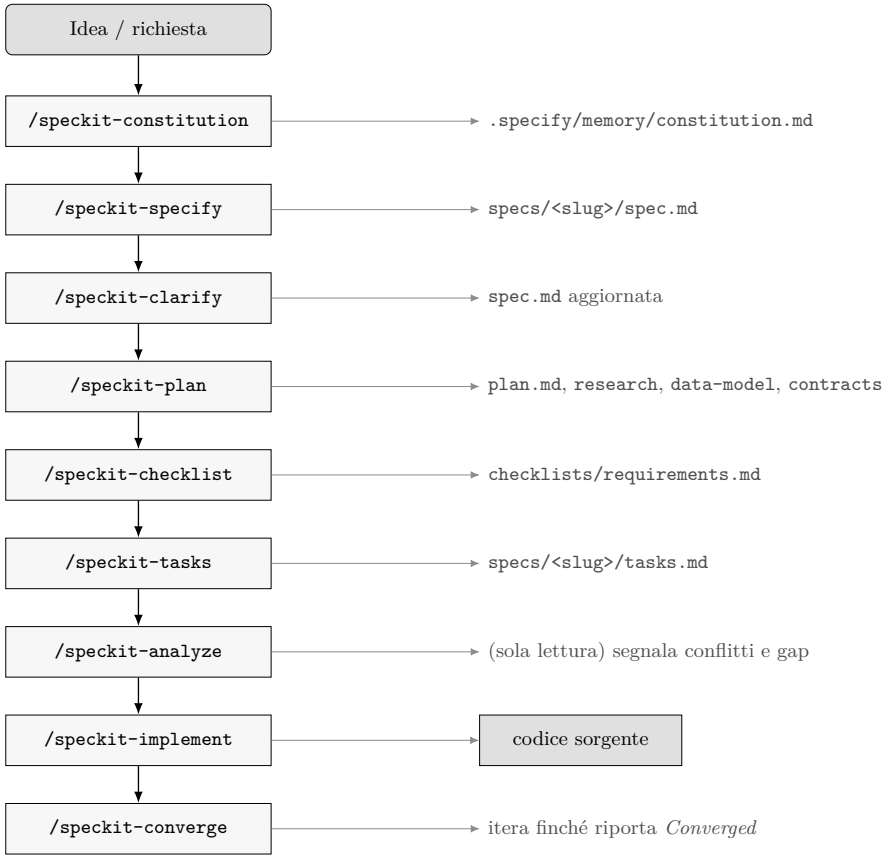


Figura 2: Il flusso completo di Spec Kit: comandi a sinistra, artefatti prodotti a destra

3.5.4 Le regole d'uso

Tre regole fondamentali, ripetute nella documentazione ufficiale:

1. **Un comando alla volta.** Non incatenare i comandi: esegui `/speckit-specify`, rivedi il risultato, poi passa al successivo. Ogni comando dipende dall'output del precedente.
2. **Rivedi prima di proseguire.** Gli artefatti sono tuoi: se la specifica non riflette ciò che volevi, correggila (a mano o chiedendo all'agente) prima di pianificare.

3. **I comandi si eseguono nella chat, non nel terminale.** Il CLI `specify` serve per `init`, estensioni e controlli dei prerequisiti; i comandi `/speckit-*` sono skill dell'agente.

3.5.5 Cosa produce la convergenza

L'ultimo comando, `/speckit-converge`, confronta il **codice** con **spec, plan e task**. Se trova gap (funzionalità richieste ma non implementate), **aggiunge nuovi task** a `tasks.md`; poi riesegui `/speckit-implement` e di nuovo `/speckit-converge`, finché non riporta **Converged**. È il meccanismo che chiude il cerchio e garantisce che il codice corrisponda alla specifica — non "più o meno", ma in modo verificabile.

3.6 La Constitution: i principi del progetto

La **constitution** è la base di tutto: un insieme di principi che governano il modo in cui le specifiche diventano codice. Vive in `.specify/memory/constitution.md` e viene letta dai comandi di piano e implementazione, che devono **conformarsi** ad essa.

3.6.1 Cosa contiene

Il template della constitution definisce una struttura chiara:

```
# [Nome progetto] Constitution

## Core Principles
### I. [Nome principio]
[Descrizione]

### II. [Nome principio]
[Descrizione]
...

## [Sezione aggiuntiva, es. Security Requirements]
[Contenuto]

## [Sezione aggiuntiva, es. Development Workflow]
[Contenuto]

## Governance
[Regole di governo: la constitution prevale sulle altre pratiche,
↪  ecc.]
```

```
**Version**: ... | **Ratified**: ... | **Last Amended**: ...
```

I **Core Principles** sono articoli numerati; le sezioni aggiuntive coprono vincoli specifici (sicurezza, performance, workflow); la sezione **Governance** stabilisce come si modifica la constitution e che essa prevale sulle altre pratiche.

3.6.2 Un esempio per un progetto .NET

Ecco una constitution realistica per il nostro progetto di esempio (la vedremo applicata nel resto del capitolo):

```
# Taskify Constitution

## Core Principles

### I. Test-First (NON NEGOZIABILE)
TDD obbligatorio: i test si scrivono prima dell'implementazione,
devono fallire, e solo dopo si implementa (Red-Green-Refactor).

### II. Semplicità
Parti dalla soluzione più semplice. Niente astrazioni
 ↪  speculative:
introduci un'astrazione solo quando serve davvero (YAGNI).

### III. Validazione degli input
Ogni input esterno deve essere validato e restituire errori
 ↪  chiari.
Nessuna eccezione non gestita verso il chiamante.

### IV. API REST coerenti
Gli endpoint usano JSON, verbi HTTP corretti e codici di stato
semantici. Le risorse sono al plurale (/tasks).
Per una transizione di dominio modellata come comando esplicito,
 ↪  è
ammesso un endpoint `POST` dedicato (per esempio
 ↪  `/tasks/{id}/complete`),
purché il piano ne motivi la scelta e ne documenti gli effetti.

## Security Requirements
- Nessun segreto nel codice o nei file di configurazione
 ↪  versionati.
- Le operazioni distruttive richiedono conferma esplicita.
```

```
## Development Workflow
- Ogni feature passa da specifica, piano e task prima del codice.
- Le modifiche sono riviste tramite pull request.

## Governance
La constitution prevale sulle altre pratiche. Può essere
 ↪  emendata, ma le
modifiche richiedono documentazione, approvazione e un piano di
 ↪  migrazione;
gli emendamenti sono versionati e aggiornano le date indicate in
 ↪  calce.

**Version**: 1.0.0 | **Ratified**: 2026-01-15 | **Last Amended**:
 ↪  2026-01-15
```

3.6.3 Come viene usata

La constitution non è un documento ornamentale. Durante il piano, il comando `/speckit-plan` esegue un **Constitution Check**: una lista di gate che il piano deve superare prima di procedere. Per esempio, con la constitution sopra, il piano dovrà dimostrare:

- che i test sono previsti **prima** dell'implementazione (Test-First);
- che l'architettura scelta è la più **semplice** (niente microservizi se basta una Minimal API);
- che la **validazione** degli input è prevista;
- che le API seguono le **convenzioni REST** dichiarate.

Se il piano viola un principio (per esempio introduce un'astrazione ingiustificata), deve **giustificarlo** esplicitamente nella sezione "Complexity Tracking", oppure semplificare.

3.6.4 La creazione con `/speckit-constitution`

Il comando si invoca **una volta per progetto**, con una descrizione dei principi voluti:

```
/speckit-constitution Taskify è un progetto "security-first":
 ↪  tutti gli input
vanno validati. Usiamo una Minimal API .NET con SQLite. I test
 ↪  sono obbligatori
e vanno scritti prima del codice.
```

L'agente produce (o aggiorna) `.specify/memory/constitution.md` traducendo la descrizione in principi strutturati. Puoi poi **modificarla a mano**: è un normale file Markdown versionato.

3.6.5 Perché è potente

La constitution è ciò che rende il comportamento dell'agente **coerente nel tempo e tra i membri del team**. Senza di essa, ogni feature potrebbe essere implementata con scelte diverse; con essa, tutte le feature rispettano gli stessi principi, e le violazioni devono essere esplicite e motivate. In un'organizzazione, è il modo per codificare gli standard architetturali e farli rispettare — non per buona volontà, ma per **gate**.

3.7 `/speckit-specify`: dalla richiesta alla specifica

Il primo comando del flusso trasforma una descrizione in linguaggio naturale in una **specifica strutturata**. È il momento in cui l'idea vaga diventa un documento con requisiti, scenari e criteri di successo.

3.7.1 Come si invoca

```
/speckit-specify Crea una Task Manager API dove gli utenti
 ↪  possono creare task,
elencarli con un filtro per stato, completarli ed eliminarli.
```

Il comando:

1. determina il nome e il numero della feature (per esempio `001-task-manager`);
2. crea la cartella `specs/001-task-manager/`;
3. copia il **template di specifica** in `spec.md` e lo compila con i requisiti estratti dalla richiesta;
4. genera la **checklist di qualità** in `checklists/requirements.md`.

3.7.2 La struttura di `spec.md`

Il template di specifica è il cuore del comando. Le sue sezioni principali:

Sezione	Scopo
User Scenarios & Testing	User story prioritarie (P1, P2, …), ciascuna con test indipendente e scenari Given/When/Then
Edge Cases	Condizioni limite e scenari d'errore
Requirements	Requisiti funzionali `FR-xxx` + entità chiave
Success Criteria	Criteri di successo misurabili `SC-xxx`, indipendenti dalla tecnologia
Assumptions	Ipotesi ragionevoli quando la richiesta non specifica dettagli

3.7.3 Le user story prioritarie e indipendenti

La sezione più importante è quella delle user story. Il template impone due proprietà fondamentali:

1. **Priorità** — ogni story ha una priorità (`P1`, `P2`, `P3`…), con `P1` la più critica.
2. **Testabilità indipendente** — ogni story deve poter essere sviluppata, testata e dimostrata **da sola**: se implementi solo la `P1`, hai comunque un MVP che porta valore.

Per ogni story il template chiede:

- una descrizione in linguaggio semplice;
- il **motivo della priorità**;
- un **test indipendente** (come verificarla da sola);
- gli **acceptance scenarios** in formato **Given/When/Then**.

3.7.4 L'esempio C#: la specifica

Applichiamo subito il template alla nostra feature. Ecco come appare `specs/001-task-manager/spec.md` compilata (mostriamo le parti essenziali):

```
# Feature Specification: Task Manager API

**Feature Branch**: `001-task-manager`
```

```
**Status**: Draft
**Input**: User description: "Crea una Task Manager API dove gli
 ↪  utenti
possono creare task, elencarli con un filtro per stato,
 ↪  completarli ed
eliminarli."

## User Scenarios & Testing

### User Story 1 - Creare un task (Priority: P1)

Un utente crea un task fornendo un titolo e, facoltativamente,
 ↪  una data di
scadenza. Il task viene salvato e restituito con il suo
 ↪  identificativo.

**Why this priority**: senza la creazione non esiste alcun dato
 ↪  su cui
operare: è il fondamento dell'intera feature.

**Independent Test**: si può verificare con una singola chiamata
POST /tasks e la successiva lettura del task creato.

**Acceptance Scenarios**:
1. **Given** nessun task esistente, **When** l'utente invia POST
 ↪  /tasks
   con un titolo valido, **Then** il task viene creato e
 ↪  restituito con id.
2. **Given** una richiesta senza titolo, **When** l'utente invia
 ↪  POST /tasks,
   **Then** il sistema risponde 400 con un messaggio di
 ↪  validazione.

---

### User Story 2 - Elencare e filtrare i task (Priority: P2)

L'utente elenca i task, con un filtro opzionale per stato
 ↪  (open/completed).

**Why this priority**: la creazione è inutile senza la
 ↪  possibilità di
consultare i task.

**Independent Test**: si verifica con GET /tasks e GET
 ↪  /tasks?status=...
```

```
**Acceptance Scenarios**:
1. **Given** task esistenti in stati diversi, **When** l'utente
 ↪  richiede
   GET /tasks?status=open, **Then** riceve solo i task aperti.

---

### User Story 3 - Completare ed eliminare (Priority: P3)

L'utente marca un task come completato o lo elimina.

**Why this priority**: chiude il ciclo di vita, ma il sistema è
 ↪  già utile
con le prime due story.

**Acceptance Scenarios**:
1. **Given** un task esistente, **When** l'utente invia
   POST /tasks/{id}/complete, **Then** il task risulta
 ↪  completato.
2. **Given** un task esistente, **When** l'utente invia
   DELETE /tasks/{id}, **Then** il task viene eliminato.

### Edge Cases

- Cosa succede se il titolo è vuoto o supera 200 caratteri?
- Cosa succede se si richiede un task con id inesistente?
- Cosa succede se la data di scadenza è nel passato?

## Requirements

### Functional Requirements

- **FR-001**: Il sistema DEVE permettere di creare un task con
 ↪  titolo
  obbligatorio (max 200 caratteri) e data di scadenza opzionale.
- **FR-002**: Il sistema DEVE elencare i task, con filtro
 ↪  opzionale per stato.
- **FR-003**: Il sistema DEVE permettere di marcare un task come
 ↪  completato.
- **FR-004**: Il sistema DEVE permettere di eliminare un task.
- **FR-005**: Il sistema DEVE restituire errori chiari in caso di
 ↪  input
  non valido o risorsa inesistente.
```

```
### Key Entities

- **Task**: un'attività con titolo, stato (aperto/completato),
  data di scadenza opzionale e data di creazione.

## Success Criteria

- **SC-001**: una task può essere creata e letta in meno di 100
 ↪  ms in locale.
- **SC-002**: il 100% degli scenari di accettazione è coperto da
 ↪  test.
- **SC-003**: nessuna richiesta valida produce un errore 500.

## Assumptions

- Il primo rilascio non prevede autenticazione: l'API è
 ↪  monoutente.
- La persistenza usa un database locale SQLite.
- La paginazione è fuori scope per la v1.
```

3.7.5 I punti chiave del template

Osserva come il template **vincola** il modello:

- i requisiti sono `FR-xxx` numerati e verificabili, non prosa libera;
- le incertezze sono esplicitate (il template prevede il marcatore `[NEEDS CLARIFICATION]` per i requisiti non specificati: lo vedremo con `/speckit-clarify`);
- i criteri di successo sono **misurabili e indipendenti dalla tecnologia** (nessun "usa .NET": quello appartiene al piano, non alla specifica);
- le entità sono descritte **senza dettagli implementativi** ("titolo, stato, scadenza", non "tabella SQL con colonna VARCHAR(200)").

Questa separazione tra *cosa* (specifica) e *come* (piano) è il principio più importante dell'intero metodo.

3.8 `/speckit-clarify`: risolvere le ambiguità

Dopo la specifica, quasi sempre restano punti oscuri. Il comando `/speckit-clarify` li affronta sistematicamente, prima che diventino decisioni arbitrarie (o bug) in fase di implementazione.

3.8.1 Cosa fa

```
/speckit-clarify Concentrati sul comportamento dei task:
 ↪  validazione del titolo,
gestione degli id inesistenti e semantica del filtro per stato.
```

Il comando:

1. esamina `spec.md` alla ricerca di ambiguità e punti deboli;
2. **pone domande** all'utente (una alla volta, con opzioni o risposte libere);
3. aggiorna `spec.md` con le risposte;
4. aggiorna la checklist dei requisiti in `checklists/requirements.md`.

È un dialogo, non un comando "one-shot": l'agente fa domande, tu rispondi, e la specifica si affina.

3.8.2 Il marcatore `[NEEDS CLARIFICATION]`

Il template di specifica prevede di marcare esplicitamente ciò che non è stato specificato. Per esempio:

```
- **FR-006**: Il sistema DEVE autenticare gli utenti tramite
  [NEEDS CLARIFICATION: metodo non specificato - email/password,
   ↪  SSO, OAuth?]
```

`/speckit-clarify` è il momento in cui questi marcatori vengono **risolti**: la domanda viene posta, la risposta sostituisce il marcatore. L'alternativa — lasciare che l'agente inventi una risposta plausibile — è esattamente ciò che il metodo vuole evitare.

3.8.3 Esempi di domande per la nostra feature

Per la Task Manager API, domande tipiche di chiarimento:

- "Il titolo deve avere un limite di lunghezza? Se sì, quale?"
- "La data di scadenza può essere nel passato?"
- "Il filtro per stato accetta solo `open`/`completed`, o anche valori parziali?"
- "L'eliminazione deve essere fisica (record rimosso) o logica (soft delete)?"
- "Serve una data di completamento registrata quando un task viene completato?"

Ogni risposta si traduce in un requisito aggiornato o in una nuova voce nella sezione Edge Cases/Requirements.

3.8.4 Quando usarlo

- **Sempre** nel percorso completo (per feature di produzione);
- per feature piccole e ben definite si può saltare, come previsto dal percorso breve;
- in ogni caso, se la specifica contiene `[NEEDS CLARIFICATION]` irrisolti, è un segnale che serve un giro di chiarimento.

3.8.5 Il valore per il team

`/speckit-clarify` sposta le decisioni di prodotto **dove costano meno**: prima di pianificare e molto prima di scrivere codice. Una domanda risolta nella specifica vale dieci correzioni in code review. E poiché le risposte finiscono nel `spec.md` versionato, restano documentate per chiunque rilegga la feature in futuro.

3.9 `/speckit-plan`: il piano tecnico

Ora che *cosa* costruire è chiaro, `/speckit-plan` decide *come*. È il comando che introduce — finalmente — la tecnologia, nel nostro caso **C# e .NET**.

3.9.1 Come si invoca

```
/speckit-plan Usa .NET 10 con una Minimal API ASP.NET Core.
 ↪  Persistenza con
Entity Framework Core su SQLite. Test con xUnit.
```

La richiesta di piano fornisce i **vincoli tecnici** (stack, librerie, piattaforma). Se qualcosa non è specificato, il comando lo chiede o lo marca `NEEDS CLARIFICATION`.

3.9.2 Le fasi del piano

Il comando lavora in fasi:

- **Fase 0 — Ricerca.** Se serve, esplora le opzioni tecniche (librerie, pattern) e produce `research.md`.

- **Fase 1 — Progettazione.** Produce `data-model.md` (le entità), `contracts/` (gli endpoint e i loro contratti), `quickstart.md` (gli scenari chiave di validazione).
- **Fase 2 — Piano.** Produce `plan.md`, il documento principale.

In più, **prima della Fase 0** esegue il **Constitution Check**: il piano deve rispettare i principi della constitution, oppure giustificare le violazioni.

3.9.3 La struttura di `plan.md`

Il template di piano ha queste sezioni:

Sezione	Contenuto
Summary	Requisito primario + approccio tecnico scelto
Technical Context	Linguaggio, dipendenze, storage, test, piattaforma, performance, vincoli, scala
Constitution Check	I gate derivati dalla constitution
Project Structure	Albero delle cartelle del codice sorgente
Complexity Tracking	(solo se serve) giustificazione delle violazioni di semplicità

3.9.4 L'esempio C#: il piano

Ecco `specs/001-task-manager/plan.md` compilata per la nostra feature:

```
# Implementation Plan: Task Manager API

**Branch**: `001-task-manager` | **Spec**:
 ↪  specs/001-task-manager/spec.md

## Summary

Una Minimal API ASP.NET Core che espone operazioni CRUD su task
(creazione, elenco con filtro, completamento, eliminazione), con
persistenza su SQLite tramite Entity Framework Core.

## Technical Context
```

```
**Language/Version**: C# 14 / .NET 10
**Primary Dependencies**: ASP.NET Core (Minimal API),
  Microsoft.EntityFrameworkCore.Sqlite
**Storage**: SQLite (file locale)
**Testing**: xUnit + WebApplicationFactory
**Target Platform**: server Windows/Linux/macOS
**Project Type**: web-service
**Performance Goals**: <100 ms per operazione su set di 10.000
 ↪  task
**Constraints**: nessun servizio esterno, API REST JSON
**Scale/Scope**: monoutente, migliaia di task

## Constitution Check

- [x] Test-First: i test di accettazione precedono
 ↪  l'implementazione
- [x] Semplicità: una sola Minimal API, nessuna astrazione
 ↪  superflua
- [x] Validazione input: DataAnnotations/minimal validation sugli
 ↪  endpoint
- [x] API REST coerenti: risorse al plurale (/tasks), codici di
 ↪  stato semantici; il completamento usa il comando esplicito
 ↪  `POST /tasks/{id}/complete`, ammesso dalla constitution

## Project Structure

src/
/-- Taskify.csproj
/-- Program.cs            # bootstrap + endpoint
/-- Models/
/   `-- TaskItem.cs
/-- Data/
/   `-- TaskDbContext.cs
`-- Contracts/
    `-- TaskDtos.cs

tests/
`-- Taskify.Tests/
    /-- Taskify.Tests.csproj
    `-- TaskEndpointsTests.cs

**Structure Decision**: progetto singolo con cartella tests/
 ↪  separata.
```

3.9.5 Il modello dati e i contratti

Il piano produce anche due artefatti preziosi. `data-model.md`:

```
# Data Model: Task Manager API

## TaskItem

| Campo | Tipo | Vincoli |
|-------|------|---------|
| Id | int | chiave primaria |
| Title | string | obbligatorio, max 200 |
| IsCompleted | bool | default false |
| DueDate | DateTime? | opzionale |
| CreatedAt | DateTimeOffset | impostato dall'applicazione in UTC |

Relazioni: nessuna (entità singola per la v1).
```

E `contracts/tasks-api.md` con i contratti degli endpoint:

```
## POST /tasks
Request:  { "title": "string", "dueDate": "date-time?" }
Response: 201 Created, body: { "id": 1, "title": "...", ... }

## GET /tasks?status=open|completed
Response: 200 OK, body: [ { ...TaskItem... } ]

## POST /tasks/{id}/complete
Response: 200 OK oppure 404 Not Found

È un endpoint di comando per la transizione esplicita di stato `complete`,
ammessa dal principio API REST della constitution; non crea una nuova task.

## DELETE /tasks/{id}
Response: 204 No Content oppure 404 Not Found
```

3.9.6 Perché il piano è verificato rispetto alla constitution

Nota la sezione **Constitution Check** del piano: non è un commento, è un **gate**. Se, per esempio, il piano avesse proposto una soluzione a microservizi per una feature monoutente, la constitution (principio di Semplicità) avrebbe imposto di giustificarlo o semplificare. Il piano, quindi,

non è una libera invenzione dell'agente: è un'interpretazione **vincolata** dei principi del progetto.

3.10 `/speckit-checklist`: i gate di qualità della specifica

Prima di scomporre il lavoro, Spec Kit offre un controllo di qualità dedicato alla specifica: `/speckit-checklist`. È il modo per assicurarsi che i requisiti siano **completi, chiari e coerenti** prima di impegnarsi nell'implementazione.

3.10.1 Cosa fa

```
/speckit-checklist
```

Il comando genera una checklist aggiuntiva e mirata al controllo richiesto (per esempio `checklists/api-quality.md`), distinta dalla checklist dei requisiti prodotta dai comandi di specifica. È una sorta di "unit test per i requisiti" e resta **di proprietà del revisore** (reviewer-owned): le caselle vanno spuntate `[x]` solo quando un revisore (umano o agente, ma esplicitamente in veste di revisore) stabilisce che il criterio è soddisfatto.

> **Importante.** Spuntare una voce della checklist **non** significa che l'implementazione è completa: significa che il *requisito* ha superato il controllo di qualità. L'implementazione è tracciata altrove (in `tasks.md`).

3.10.2 Un esempio di checklist

Per la nostra feature, una checklist aggiuntiva di qualità dell'API potrebbe contenere voci come:

```
# API Quality Checklist

## Completezza
- [ ] Ogni user story ha priorità, test indipendente e scenari di
 ↪  accettazione
- [ ] Tutti i requisiti funzionali sono numerati (FR-xxx)
- [ ] Tutti gli edge case rilevanti sono elencati

## Chiarezza
- [ ] Nessun requisito usa un linguaggio ambiguo ("veloce",
 ↪  "robusto")
```

```
- [ ] Ogni requisito è verificabile in modo oggettivo
- [ ] Nessun dettaglio implementativo nella specifica

## Coerenza
- [ ] I criteri di successo sono misurabili (SC-xxx)
- [ ] Le entità chiave coprono i dati menzionati dalle story
- [ ] Non ci sono contraddizioni tra story e requisiti
```

3.10.3 Chi mantiene cosa

Spec Kit distingue la checklist di requisiti generata nel flusso di specifica dalle checklist aggiuntive:

- `checklists/requirements.md` — prodotta da `/speckit-specify` e aggiornata da `/speckit-clarify`; è il gate letto da `/speckit-implement`.
- **checklist aggiuntive** — generate da `/speckit-checklist` per un ambito specifico, con un nome distinto (per esempio `checklists/api-quality.md`), e di proprietà del revisore.

3.10.4 Il gate in `/speckit-implement`

La checklist non è fine a se stessa: quando esegui `/speckit-implement`, il comando **legge lo stato delle caselle** e, se trova voci non spuntate, **chiede** prima di procedere. È un semaforo: se la specifica non ha superato il controllo, non si costruisce sopra una base incerta.

3.10.5 Il valore

In un flusso classico, i problemi di requisiti emergono durante lo sviluppo o, peggio, in produzione. Con la checklist, emergono **prima di scrivere una riga di codice**, quando correggerli costa quasi nulla. È lo stesso principio dei test automatici, applicato ai requisiti.

3.11 `/speckit-tasks`: la scomposizione in task

A questo punto la specifica è chiara e il piano tecnico definito. `/speckit-tasks` trasforma tutto in una **lista di task eseguibili e ordinati per dipendenza**.

3.11.1 Come si invoca

```
/speckit-tasks
```

Nessun argomento: il comando legge `plan.md` (obbligatorio), `spec.md` (per le user story) e, se presenti, `research.md`, `data-model.md` e `contracts/`, e produce `tasks.md`.

3.11.2 Il formato dei task

Il template impone un formato compatto:

```
- [ ] T001 [P] [US1] Creare il progetto e aggiungere i pacchetti
 ↪  in src/
```

Dove:

- `T001` — identificativo progressivo del task;
- `[P]` — opzionale, indica che il task può essere eseguito **in parallelo** (file diversi, nessuna dipendenza);
- `[US1]` — la **user story** a cui appartiene il task;
- la descrizione include **i percorsi file esatti** da toccare.

3.11.3 Le fasi

I task sono raggruppati in **fasi**, in quest'ordine:

1. **Phase 1: Setup** — inizializzazione del progetto (struttura, dipendenze, linting).
2. **Phase 2: Foundational** — infrastruttura **bloccante** che deve esistere prima di ogni user story (schema del database, routing, middleware, configurazione). Nessuna user story può partire prima di questa fase.
3. **Phase 3+: User Story N** — un gruppo di task per ogni user story, in ordine di priorità (P1, P2, …). Ogni gruppo è **indipendente**: si può implementare e testare la P1 da sola.

3.11.4 L'esempio C#: i task

Ecco `specs/001-task-manager/tasks.md` (parti essenziali):

```
# Tasks: Task Manager API

**Input**: design da specs/001-task-manager/
```

```
## Phase 1: Setup

- [ ] T001 Creare il progetto web `src/Taskify.csproj` e
↪ aggiungere EF Core Sqlite
- [ ] T002 Creare `tests/Taskify.Tests/Taskify.Tests.csproj` con
↪ xUnit, aggiungere il riferimento a `src/Taskify.csproj` e il
↪ pacchetto `Microsoft.AspNetCore.Mvc.Testing`

## Phase 2: Foundational (Bloccante)

- [ ] T003 Creare l'entità TaskItem in src/Models/TaskItem.cs
- [ ] T004 Creare TaskDbContext in src/Data/TaskDbContext.cs
- [ ] T005 Creare i DTO in src/Contracts/TaskDtos.cs
- [ ] T006 Configurare EF Core e il database in src/Program.cs,
↪ rendendo `Program` accessibile ai test

**Checkpoint**: fondamenta pronte - le user story possono partire
↪ in parallelo.

## Phase 3: User Story 1 - Creare un task (P1) [MVP]

**Goal**: POST /tasks crea e persiste un task.
**Independent Test**: una chiamata POST /tasks seguita da GET
↪ /tasks/{id}.

- [ ] T007 [US1] Scrivere il test per POST /tasks in
↪ tests/Taskify.Tests/TaskEndpointsTests.cs
- [ ] T008 [US1] Eseguire il test e verificare che fallisca (RED)
- [ ] T009 [US1] Implementare l'endpoint POST /tasks in
↪ src/Program.cs
- [ ] T010 [US1] Verificare che il test passi (GREEN)

## Phase 4: User Story 2 - Elencare e filtrare (P2)

- [ ] T011 [US2] Scrivere il test per GET /tasks?status=... in
↪ tests/Taskify.Tests/TaskEndpointsTests.cs
- [ ] T012 [US2] Eseguire il test e verificare che fallisca (RED)
- [ ] T013 [US2] Implementare GET /tasks con filtro in
↪ src/Program.cs
- [ ] T014 [US2] Verificare che il test passi (GREEN)

## Phase 5: User Story 3 - Completare ed eliminare (P3)

- [ ] T015 [US3] Scrivere i test per POST /tasks/{id}/complete e
↪ DELETE /tasks/{id} in
↪ tests/Taskify.Tests/TaskEndpointsTests.cs
```

```
- [ ] T016 [US3] Eseguire i test e verificare che falliscano
 ↪  (RED)
- [ ] T017 [US3] Implementare i due endpoint in src/Program.cs
- [ ] T018 [US3] Verificare che i test passino (GREEN)
```

3.11.5 Il test-first nei task

Nota la struttura di ogni gruppo: **prima i test, poi la verifica RED, poi l'implementazione e la verifica GREEN**. È la constitution (principio Test-First) che si riflette direttamente nella forma dei task.

3.11.6 Perché i task funzionano

Un task ben formato è:

- **atomico** — una sola responsabilità, un solo file (o pochi);
- **ordinato** — le fasi impongono l'ordine; `[P]` segnala il parallelismo possibile;
- **tracciabile** — ogni task è legato a una user story, quindi a un requisito, quindi alla specifica;
- **verificabile** — ogni gruppo ha un "Independent Test" che ne conferma il completamento.

È questa tracciabilità a rendere il flusso Spec Kit molto più di una lista di cose da fare: è una **catena di responsabilità** che va dall'idea al codice.

3.12 `/speckit-analyze`: la verifica di coerenza

Prima di implementare, Spec Kit offre un controllo trasversale: `/speckit-analyze` verifica che spec, piano e task siano **coerenti tra loro**.

3.12.1 Cosa fa

```
/speckit-analyze
```

Il comando è **di sola lettura**: non modifica nulla. Esamina `spec.md`, `plan.md` e `tasks.md` e segnala:

- **conflitti** — per esempio un requisito nella specifica che il piano non copre, o due requisiti in contraddizione;

- **gap** — una user story della specifica che non ha task corrispondenti, o un endpoint nel piano senza task;
- **ambiguità** — marcatori `[NEEDS CLARIFICATION]` rimasti irrisolti.

3.12.2 Esempi di segnalazioni

Per la nostra feature, l'analisi potrebbe segnalare:

```
[GAP] FR-004 ("eliminare un task") non ha task associati in
↪  tasks.md.
[GAP] L'edge case "data di scadenza nel passato" è nella
↪  specifica ma non
     è coperto da alcun requisito di validazione nel piano.
[CONFLITTO] Il piano usa /tasks/{id}/complete ma la specifica
↪  menziona
     "marcare come completato" senza specificare il metodo HTTP.
```

3.12.3 Cosa fare con le segnalazioni

La regola è netta: **si correggono gli artefatti alla fonte**, non si forza l'implementazione. Se l'analisi trova un gap, aggiorni `tasks.md` (o la specifica, o il piano), poi riesegui `/speckit-analyze` finché non è pulito. Implementare sopra un'analisi sporca è esattamente ciò che il gate vuole evitare.

3.12.4 Il posto nel flusso

Nel percorso completo, `/speckit-analyze` viene **dopo** `/speckit-tasks` e **prima** di `/speckit-implement`. È l'ultimo controllo prima che l'agente inizi a scrivere codice: il costo di una correzione qui è minimo, mentre il costo dello stesso errore scoperto dopo l'implementazione è alto.

3.12.5 La filosofia

`/speckit-analyze` incarna una convinzione profonda di Spec Kit: gli artefatti (spec, plan, task) non sono documenti separati, ma **viste dello stesso sistema**. Devono restare allineati. Quando un agente genera molto codice in fretta, l'allineamento è la prima cosa a rompersi; un controllo automatico di coerenza è la rete di sicurezza che lo preserva.

3.13 Implement e Converge

Siamo al momento in cui l'agente scrive il codice. Gli ultimi due comandi — implement e converge — lavorano in coppia e chiudono il ciclo.

3.13.1 `/speckit-implement`

```
/speckit-implement
```

Il comando esegue i task di `tasks.md` **in ordine di dipendenza** (fase per fase). Prima di partire:

1. **legge la checklist** (`checklists/requirements.md`): se ci sono voci non spuntate, **chiede** all'utente prima di procedere (senza modificare i file di checklist, che restano di proprietà del revisore);
2. rispetta i gate della constitution definiti nel piano.

Per feature grandi, si può eseguire **una fase alla volta** (per esempio solo la Phase 3), per controllare meglio l'avanzamento. Per feature piccole, una singola esecuzione costruisce tutto.

3.13.2 Cosa produce

L'output è il **codice sorgente vero e proprio**: nel nostro caso, i file C# della Minimal API, l'entità, il `DbContext`, i DTO e i test — esattamente i percorsi indicati nei task. Lo vedremo in dettaglio nella sezione dell'esempio completo.

3.13.3 `/speckit-converge`

```
/speckit-converge
```

Dopo l'implementazione, converge **confronta il codice con spec, plan e task**:

- se trova funzionalità richieste ma non implementate (gap), **aggiunge nuovi task** a `tasks.md`;
- altrimenti riporta **Converged**.

Se ci sono gap, il ciclo è: esegui di nuovo `/speckit-implement` per i nuovi task, poi di nuovo `/speckit-converge`, finché non riporta `Converged`.

3.13.4 Il ciclo implement–converge

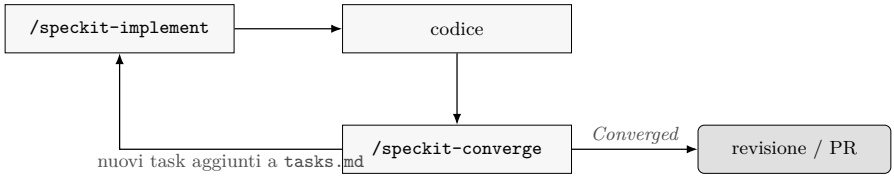


Figura 3: Il ciclo implement–converge

Quando converge riporta `Converged`, il lavoro è completo: puoi procedere con la revisione umana o aprire la pull request.

3.13.5 Cosa garantisce la convergenza

La convergenza è ciò che trasforma "l'agente ha scritto del codice" in "l'agente ha scritto **il** codice richiesto". È una verifica di **completezza**, non di correttezza: i test (scritti in modalità test-first) coprono la correttezza; la convergenza copre la completezza. Insieme, danno una ragionevole fiducia che il risultato corrisponda alla specifica.

3.13.6 Il risultato finale

Al termine del flusso completo hai:

- una specifica rivista (`spec.md`);
- un piano tecnico con modello dati e contratti;
- una lista di task eseguiti;
- il codice sorgente;
- i test;
- la conferma di convergenza.

E tutto è **versionato**: chiunque riapra la feature — un collega, un revisore, o te stesso tra sei mesi — trova non solo il codice, ma il *perché* e il *come* documentati.

3.14 Il CLI `specify`

I comandi `/speckit-*` vivono nella chat dell'agente, ma Spec Kit include anche un **CLI** (`specify`) per le operazioni di terra: inizializzazione, controlli dei prerequisiti, gestione di estensioni, preset e workflow.

3.14.1 I comandi principali

Comando	Scopo
`specify init`	Inizializza un progetto (già visto)
`specify check`	Controlla la disponibilità degli strumenti e dei prerequisiti CLI
`specify extension add <nome>`	Installa un'estensione (per esempio `bug`, `assess`)
`specify preset search/apply`	Cerca e applica preset della comunità
`specify workflow search/run`	Cerca ed esegue workflow
`specify bundle search/install`	Cerca e installa bundle

3.14.2 `specify check`

Il comando `specify check` controlla la disponibilità degli strumenti e dei prerequisiti richiesti dal CLI nella versione installata. Non legge né convalida la coerenza di `spec.md`, `plan.md`, `tasks.md` o degli altri artefatti; per questi servono la revisione e i controlli del workflow Spec Kit (per esempio `/speckit-analyze` e `/speckit-converge`).

```
specify check
```

Se manca un prerequisito, il comando segnala il problema e può restituire un codice di uscita diverso da zero. `specify self check`, se disponibile nella versione installata, è un'autodiagnosi del CLI; non è un validatore degli artefatti del progetto.

3.14.3 Le estensioni: bug fixing e assessment

Abbiamo accennato ai due processi opzionali. Si installano come estensioni:

```
specify extension add bug
specify extension add assess
```

Bug fixing aggiunge i comandi:

```
/speckit-bug-assess "Sottomettere una password vuota manda in
↪  crash il form." slug=login-crash
/speckit-bug-fix slug=login-crash
/speckit-bug-test slug=login-crash
```

Gli artefatti finiscono in `.specify/bugs/<slug>/`, e il verdetto finale è `verified`, `partial` o `failed`: solo `verified` conferma la correzione.

Idea assessment aggiunge i comandi:

```
/speckit-assess-intake "Consenti agli utenti di lavorare
↪  offline." slug=offline-mode
/speckit-assess-research slug=offline-mode
/speckit-assess-define slug=offline-mode
/speckit-assess-shape slug=offline-mode
/speckit-assess-decide slug=offline-mode
```

Gli artefatti finiscono in `.specify/assessments/<slug>/`, e la decisione finale è `go`, `needs-clarification` o `kill`. Se l'idea ottiene un `go`, il riepilogo di `decision.md` diventa l'input di `/speckit-specify`.

3.14.4 Preset e workflow

- Un **preset** adatta template, comandi e terminologia agli standard del tuo team (per esempio "richiedi tracciabilità normativa nei piani").
- Un **workflow** automatizza sequenze di prompt, comandi, gate e task di shell: per esempio, un workflow CI che compila ed esegue i test del progetto. `specify check` può servire durante la configurazione dell'ambiente, ma non è un gate di qualità degli artefatti.
- Un **bundle** impacchetta estensioni, preset e workflow per distribuirli a un team o a un ruolo.

3.14.5 Estensioni e hook

Il file `.specify/extensions.yml` può registrare **hook**, cioè azioni da eseguire prima o dopo i comandi. Un caso tipico: un hook `before_specify` che crea (o cambia) il **branch Git** all'avvio di una feature. Questo è ciò che permette di integrare Spec Kit con le abitudini del team (feature branch numerati, `001-task-manager`, ecc.), senza che il flusso ne dipenda: come abbiamo visto, la feature attiva è determinata da `.specify/feature.json`, non dal branch.

3.14.6 Quando usare il CLI

- `init` — una volta per progetto;
- `check` — per diagnosticare la disponibilità dei prerequisiti, non per convalidare gli artefatti;
- `extension add` — quando attivi un processo opzionale;
- `preset`/`workflow`/`bundle` — quando adatti Spec Kit al tuo team.

Il resto del lavoro quotidiano avviene nella chat dell'agente, con i comandi `/speckit-*`.

3.15 Esempio C#: entità, DbContext e DTO

È il momento di vedere il codice che `/speckit-implement` produce seguendo il piano. Partiamo dalle fondamenta: l'entità, il contesto EF Core e i DTO. Corrispondono ai task `T003`, `T004` e `T005` della fase "Foundational".

3.15.1 L'entità `TaskItem`

File `src/Models/TaskItem.cs`, come stabilito dal piano:

```
namespace Taskify.Models;

public sealed class TaskItem
{
    public int Id { get; set; }
    public required string Title { get; set; }
    public bool IsCompleted { get; set; }
    public DateTime? DueDate { get; set; }
    public DateTimeOffset CreatedAt { get; set; }
}
```

È il riflesso diretto del modello dati definito in `data-model.md`: un identificativo, un titolo obbligatorio, lo stato, la scadenza opzionale e la data di creazione. Nessuna sorpresa: l'entità **è** il modello dati, tradotto in C#.

3.15.2 Il `DbContext`

File `src/Data/TaskDbContext.cs`:

```
using Microsoft.EntityFrameworkCore;
using Taskify.Models;
```

```
namespace Taskify.Data;

public sealed class
 ↪  TaskDbContext(DbContextOptions<TaskDbContext> options)
    : DbContext(options)
{
    public DbSet<TaskItem> Tasks => Set<TaskItem>();

    protected override void OnModelCreating(ModelBuilder
     ↪  modelBuilder)
    {
        modelBuilder.Entity<TaskItem>(entity =>
        {
            entity.Property(t =>
 ↪  t.Title).HasMaxLength(200).IsRequired();
            entity.Property(t =>
 ↪  t.IsCompleted).HasDefaultValue(false);
        });
    }
}
```

Due scelte qui derivano **direttamente dalla specifica**:

- `HasMaxLength(200)` — il vincolo "titolo massimo 200 caratteri" (FR-001) non vive solo nella validazione dell'endpoint, ma anche nello schema del database: difesa in profondità;
- `HasDefaultValue(false)` — un task nasce "aperto".

3.15.3 I DTO

File `src/Contracts/TaskDtos.cs`:

```
using Taskify.Models;

namespace Taskify.Contracts;

public sealed record CreateTaskRequest(string? Title, DateTime?
 ↪  DueDate);

public sealed record TaskDto(
    int Id,
    string Title,
    bool IsCompleted,
    DateTime? DueDate,
```

```
    DateTimeOffset CreatedAt)
{
    public static TaskDto From(TaskItem task) => new(
        task.Id,
        task.Title,
        task.IsCompleted,
        task.DueDate,
        task.CreatedAt);
}
```

Il DTO separa il **contratto esterno** (ciò che l'API espone) dal **modello interno** (l'entità EF Core). È la pratica del Capitolo 2 applicata qui: mai esporre le entità grezze, controllare sempre ciò che viaggia sul filo.

3.15.4 La tracciabilità spec → codice

Nota come ogni decisione di codice sia riconducibile a un requisito:

Codice	Requisito di origine
`Title` obbligatorio, max 200	FR-001
`IsCompleted` con default `false`	FR-003 (completamento)
`DueDate?` opzionale	FR-001
`TaskDto.From` (proiezione)	Coerenza API (constitution)

Questa tracciabilità non è un vezzo: è ciò che permette, durante la **convergenza**, di verificare che ogni requisito abbia una realizzazione nel codice.

3.15.5 La registrazione dei servizi

L'ultimo tassello "foundational" è la registrazione del `DbContext` nel container (task `T006`), che vedremo all'inizio di `Program.cs` nella prossima sezione. Con queste fondamenta pronte, il "Checkpoint" della fase Foundational è superato: le user story possono essere implementate.

3.16 Esempio C#: gli endpoint della Minimal API

Ecco il cuore dell'implementazione: `src/Program.cs`, che realizza i quattro endpoint definiti nei contratti. Il listato mostra lo stato finale del

file dopo il ciclo test-first; corrisponde ai task di implementazione T009, T013 e T017. L'ordine espositivo del capitolo non è l'ordine di lavoro: i test vanno scritti ed eseguiti in RED prima di implementare gli endpoint.

3.16.1 Il codice completo

```
using Microsoft.EntityFrameworkCore;
using Taskify.Contracts;
using Taskify.Data;
using Taskify.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<TaskDbContext>(options =>
    options.UseSqlite("Data Source=tasks.db"));

var app = builder.Build();

// Crea lo schema all'avvio (solo per l'esempio; in produzione:
↪  migrazioni).
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<
    ↪  TaskDbContext>();
    db.Database.EnsureCreated();
}

// ---- US1: creare un task ----
app.MapPost("/tasks", async (CreateTaskRequest request,
↪  TaskDbContext db) =>
{
    if (string.IsNullOrWhiteSpace(request.Title))
        return Results.BadRequest(new { error = "Title is
        ↪  required." });

    if (request.Title.Length > 200)
        return Results.BadRequest(new { error = "Title must be at
        ↪  most 200 characters." });

    var task = new TaskItem
    {
        Title = request.Title,
        DueDate = request.DueDate,
        IsCompleted = false,
        CreatedAt = DateTimeOffset.UtcNow
```

```
    };

    db.Tasks.Add(task);
    await db.SaveChangesAsync();

    return Results.Created($"/tasks/{task.Id}",
     ↪  TaskDto.From(task));
});

// ---- US2: elencare e filtrare ----
app.MapGet("/tasks", async (string? status, TaskDbContext db) =>
{
    IQueryable<TaskItem> query = db.Tasks;

    if (status is not null)
    {
        var completed = status switch
        {
            "open" => false,
            "completed" => true,
            _ => (bool?)null
        };

        if (completed is null)
            return Results.BadRequest(new { error = "status must
             ↪  be 'open' or 'completed'." });

        query = query.Where(t => t.IsCompleted == completed);
    }

    var tasks = await query
        .OrderBy(t => t.Id)
        .Select(t => new TaskDto(
            t.Id,
            t.Title,
            t.IsCompleted,
            t.DueDate,
            t.CreatedAt))
        .ToListAsync();

    return Results.Ok(tasks);
});

// ---- US3: completare ----
app.MapPost("/tasks/{id:int}/complete", async (int id,
 ↪  TaskDbContext db) =>
```

```
{
    var task = await db.Tasks.FindAsync(id);
    if (task is null)
        return Results.NotFound(new { error = "Task not found."
        ↪ });

    task.IsCompleted = true;
    await db.SaveChangesAsync();

    return Results.Ok(TaskDto.From(task));
});

// ---- US3: eliminare ----
app.MapDelete("/tasks/{id:int}", async (int id, TaskDbContext db)
↪ =>
{
    var task = await db.Tasks.FindAsync(id);
    if (task is null)
        return Results.NotFound(new { error = "Task not found."
        ↪ });

    db.Tasks.Remove(task);
    await db.SaveChangesAsync();

    return Results.NoContent();
});

app.Run();

// Rende Program accessibile ai test di integrazione
↪ (WebApplicationFactory).
public partial class Program { }
```

3.16.2 Lettura guidata

La Minimal API. Abbiamo scelto il pattern *Minimal API* di ASP.NET Core: endpoint dichiarati con `MapPost`, `MapGet`, `MapDelete`, handler come lambda. È la scelta più **semplice** per una feature di questo calibro — coerente con il principio di Semplicità della constitution. Se la feature crescesse, il piano potrebbe evolvere verso i controller; per ora, minimal è la scelta giusta.

La validazione. I due controlli su `Title` (obbligatorietà e lunghezza) realizzano FR-001 e il principio di Validazione della constitution. Errori

chiari (`Title is required.`) permettono al chiamante di correggersi.

I codici di stato. Ogni risposta usa un codice semantico: `201 Created` con l'URI della risorsa, `400 Bad Request` per input non validi, `404 Not Found` per risorse inesistenti, `204 No Content` per l'eliminazione. È la coerenza REST richiesta dalla constitution.

Il binding dei parametri. ASP.NET Core associa automaticamente il corpo JSON a `CreateTaskRequest`, i segmenti di route a `id`, la query string a `status`, e inietta il `TaskDbContext` dall'DI. Nessun codice di colla.

`public partial class Program`. L'ultima riga rende la classe `Program` (generata dai top-level statements) accessibile al progetto di test, che userà `WebApplicationFactory<Program>`. È un dettaglio indispensabile per i test di integrazione.

3.16.3 Il progetto

Il progetto web si crea con:

```
> dotnet new web -o src -n Taskify
> cd src
> dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

Il file di progetto usa `Microsoft.NET.Sdk.Web`, che include implicitamente i namespaces di ASP.NET Core (per questo nel listato non servono molti `using` espliciti).

3.17 Esempio C#: i test (test-first)

La constitution impone il **Test-First**, e i task lo riflettono: prima i test, poi l'implementazione. Il listato mostra il file completo a lavoro terminato; durante il lavoro si aggiungono i test di ciascuna user story e li si esegue in RED prima di implementare la story corrispondente.

3.17.1 Il progetto di test

```
> dotnet new xunit -o tests/Taskify.Tests -n Taskify.Tests
> dotnet add tests/Taskify.Tests/Taskify.Tests.csproj reference
 ↪  src/Taskify.csproj
> dotnet add tests/Taskify.Tests/Taskify.Tests.csproj package
 ↪  Microsoft.AspNetCore.Mvc.Testing
```

Il pacchetto `Microsoft.AspNetCore.Mvc.Testing` fornisce `WebApplicationFactory<T>`, che avvia l'applicazione **in-memory** per i test di integrazione, senza bisogno di un server reale.

3.17.2 I test

File `tests/Taskify.Tests/TaskEndpointsTests.cs`:

```
using System.Net;
using System.Net.Http.Json;
using Microsoft.AspNetCore.Mvc.Testing;
using Taskify.Contracts;

namespace Taskify.Tests;

public class TaskEndpointsTests :
↪  IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public TaskEndpointsTests(WebApplicationFactory<Program>
    ↪  factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task CreateTask_WithValidTitle_ReturnsCreated()
    {
        var response = await _client.PostAsJsonAsync(
            "/tasks", new { title = "Write the tests", dueDate =
            ↪  (DateTime?)null });

        Assert.Equal(HttpStatusCode.Created,
↪   response.StatusCode);

        var task = await
        ↪  response.Content.ReadFromJsonAsync<TaskDto>();
        Assert.NotNull(task);
        Assert.Equal("Write the tests", task.Title);
        Assert.False(task.IsCompleted);
    }

    [Fact]
    public async Task CreateTask_WithoutTitle_ReturnsBadRequest()
    {
```

```
        var response = await _client.PostAsJsonAsync(
            "/tasks", new { title = "", dueDate = (DateTime?)null
            ↪  });

        Assert.Equal(HttpStatusCode.BadRequest,
↪   response.StatusCode);
    }

    [Fact]
    public async Task
     ↪  ListTasks_FilterByStatus_ReturnsOnlyOpenTasks()
    {
        await _client.PostAsJsonAsync("/tasks", new { title =
↪   "Open task" });
        await _client.PostAsJsonAsync("/tasks", new { title =
↪   "Done task" });

        var open = await _client.GetFromJsonAsync<List<TaskDto
         ↪  >>("/tasks?status=open");

        Assert.NotNull(open);
        Assert.All(open, t => Assert.False(t.IsCompleted));
    }

    [Fact]
    public async Task CompleteTask_ExistingId_ReturnsCompleted()
    {
        var created = await _client.PostAsJsonAsync("/tasks", new
         ↪  { title = "To complete" });
        var task = await
         ↪  created.Content.ReadFromJsonAsync<TaskDto>();

        var response = await
         ↪  _client.PostAsync($"/tasks/{task!.Id}/complete",
         ↪  null);

        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
        var completed = await
         ↪  response.Content.ReadFromJsonAsync<TaskDto>();
        Assert.True(completed!.IsCompleted);
    }

    [Fact]
    public async Task DeleteTask_ExistingId_ReturnsNoContent()
    {
```

```
        var created = await _client.PostAsJsonAsync("/tasks", new
        ↪  { title = "To delete" });
        var task = await
        ↪  created.Content.ReadFromJsonAsync<TaskDto>();

        var response = await
        ↪  _client.DeleteAsync($"/tasks/{task!.Id}");

        Assert.Equal(HttpStatusCode.NoContent,
↪   response.StatusCode);
    }
}
```

3.17.3 La corrispondenza con gli scenari di accettazione

Ogni test mappa uno scenario della specifica:

Test	Scenario di accettazione
`CreateTask_WithValidTitle_ReturnsCreated`	US1, scenario 1
`CreateTask_WithoutTitle_ReturnsBadRequest`	US1, scenario 2
`ListTasks_FilterByStatus_...`	US2, scenario 1
`CompleteTask_ExistingId_...`	US3, scenario 1
`DeleteTask_ExistingId_...`	US3, scenario 2

È la tracciabilità che la convergenza verificherà: ogni requisito testabile ha il suo test.

3.17.4 Il ciclo Red-Green-Refactor

Il principio Test-First impone l'ordine: scrivi il test, **verifica che fallisca** (perché l'implementazione non c'è ancora), poi implementa, poi verifica che passi. In un flusso con agente AI, questo ordine è importante: evita il "test scritto a posteriori" che verrebbe tarato sull'implementazione invece che sul requisito.

3.17.5 Eseguire i test

```
> dotnet test tests/Taskify.Tests/Taskify.Tests.csproj
```

Se tutto è in ordine, l'output riporterà i test verdi. È la conferma di **correttezza**; la conferma di **completezza** arriverà con `/speckit-converge`.

3.18 Esempio C#: eseguire, verificare e convergere

Chiudiamo l'esempio eseguendo l'applicazione, provandola con richieste reali e portando a termine la convergenza.

3.18.1 Avviare l'applicazione

```
> dotnet run --urls http://localhost:5080
```

L'applicazione parte su `http://localhost:5080`. Ora possiamo provarla con `curl.exe` da PowerShell:

3.18.2 Creare un task

```
> curl.exe -X POST "http://localhost:5080/tasks" `
    -H "Content-Type: application/json" `
    -d '{"title": "Scrivere il capitolo 3", "dueDate": null}'
```

Risposta attesa (201 Created):

```
{
  "id": 1,
  "title": "Scrivere il capitolo 3",
  "isCompleted": false,
  "dueDate": null,
  "createdAt": "2026-10-27T12:00:00+00:00"
}
```

3.18.3 Elencare con filtro

```
> curl.exe "http://localhost:5080/tasks?status=open"
```

```
[
  { "id": 1, "title": "Scrivere il capitolo 3", "isCompleted":
    ↪  false, ... }
]
```

```
> curl.exe -X POST "http://localhost:5080/tasks/1/complete"
> curl.exe -X DELETE "http://localhost:5080/tasks/1"
```

La prima risponde con il task completato, la seconda con `204 No Content`.

3.18.5 La convergenza

Apri una nuova shell nella radice del repository e verifica che i test passino (`dotnet test tests/Taskify.Tests/Taskify.Tests.csproj`); poi torniamo all'agente e invochiamo:

```
/speckit-converge
```

Il comando confronta il codice con `spec.md`, `plan.md` e `tasks.md`. Se tutto è implementato, riporta **Converged**. Se, per esempio, avessimo dimenticato la validazione della data di scadenza nel passato (un edge case della specifica), converge lo segnalerebbe e **aggiungerebbe un task** a `tasks.md`: a quel punto basterebbe rieseguire `/speckit-implement` e di nuovo `/speckit-converge`.

3.18.6 Il risultato completo

Al termine del flusso, il repository contiene:

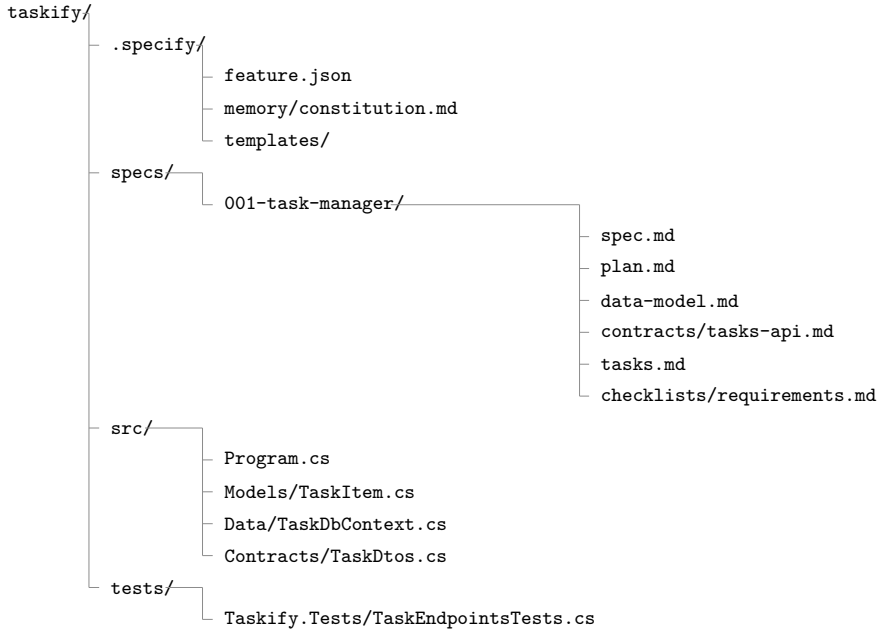


Figura 4: Struttura finale del repository dopo il flusso completo

Una feature completa: specifica, piano, task, codice, test e convergenza — tutto versionato e ricostruibile. È il percorso che abbiamo attraversato in questo capitolo, e che puoi ora ripetere per qualsiasi feature in C# (o in qualunque altro linguaggio).

3.19 Integrazione con Copilot e VS Code

Spec Kit è agente-agnostico, ma la combinazione più naturale per chi lavora in .NET è **Visual Studio Code + GitHub Copilot**. Vediamo come si presenta l'integrazione.

3.19.1 L'integrazione Copilot

Quando esegui `specify init --integration copilot`, Spec Kit registra i comandi `/speckit-*` come **skill di Copilot** nel progetto. In VS Code, questo significa che nella chat di Copilot (o nell'editor) puoi digitare:

```
/speckit-specify Crea una Task Manager API con creazione, elenco,
↪  completamento ed eliminazione.
```

e Copilot eseguirà il comando, leggendo i template e scrivendo gli artefatti in `specs/`.

3.19.2 Dove vivono le skill

A seconda della versione di Copilot, le skill vengono registrate nella cartella del progetto (per esempio in `.github/prompts/` o nelle configurazioni delle skill). Non serve preoccuparsi del dettaglio: l'importante è che, dopo `init`, i comandi `/speckit-*` siano disponibili nella chat quando il progetto è aperto.

3.19.3 Il flusso in VS Code

Il ciclo tipico in VS Code:

1. apri il progetto (`code taskify`);
2. apri la chat di Copilot;
3. invochi `/speckit-specify` con la descrizione;
4. **rileggi** `spec.md` nell'editor (i file sono lì, modificabili a mano);
5. invochi i comandi successivi uno alla volta;
6. quando arrivi a `/speckit-implement`, Copilot scrive i file C# nel progetto;
7. esegui `dotnet test` nel terminale integrato;
8. chiudi con `/speckit-converge`.

3.19.4 Altri agenti

Spec Kit supporta molti agenti: Claude Code, Cursor, Windsurf, Qoder, Amazon Q, Antigravity, Crush, Gemini CLI, Kilo Code, OpenCode, Roo Code e altri. La sintassi di invocazione cambia leggermente (ogni agente ha il suo modo di invocare una skill), ma il flusso e gli artefatti sono identici: è il valore della standardizzazione.

3.19.5 Rivedere e modificare a mano

Un punto da non sottovalutare: **gli artefatti sono file Markdown normali**. In VS Code puoi aprirli, modificarli, confrontarli con il diff di Git e rivederli come faresti con qualsiasi documento. Se la specifica

prodotta non ti convince, la correggi a mano e rilanci i comandi successivi. L'agente è un acceleratore, non un oracolo infallibile.

3.19.6 VS Code e Git

Il flusso Spec Kit si sposa naturalmente con Git: ogni feature ha la sua cartella in `specs/`, e (con l'estensione git opzionale) il suo branch. Commit e pull request diventano i punti di revisione: la specifica viene rivista e approvata **prima** che il codice venga scritto, spostando la review dove costa meno.

3.20 CI/CD e automazione

Spec Kit non si limita alla chat: la pipeline può automatizzare build e test del prodotto, mentre la revisione degli artefatti Spec Kit resta affidata al workflow dell'agente e ai revisori.

3.20.1 `specify check` non è un gate degli artefatti

`specify check` controlla la disponibilità degli strumenti e dei prerequisiti richiesti dal CLI; non convalida specifiche, piani, task o altri artefatti. È utile durante la configurazione dell'ambiente, ma non aggiunge un controllo di qualità degli artefatti alla CI. Anche `specify self check`, se disponibile, è un'autodiagnosi del CLI e non sostituisce la revisione degli artefatti.

Il comando si può eseguire localmente per diagnosticare l'ambiente. Non lo inseriamo nel workflow CI d'esempio: se fallisce, segnala un problema di prerequisiti CLI, non un artefatto Spec Kit non valido.

3.20.2 Build e test del progetto .NET

Il controllo automatico ripetibile del progetto è la build e l'esecuzione dei test. I comandi puntano esplicitamente al progetto di test perché l'esempio non presuppone una soluzione `.sln` nella radice:

```
steps:
  - uses: actions/setup-dotnet@v4
    with:
      dotnet-version: '10.0.x'
  - run: dotnet restore tests/Taskify.Tests/Taskify.Tests.csproj
  - run: dotnet build tests/Taskify.Tests/Taskify.Tests.csproj
 ↪  --no-restore
```

```
  - run: dotnet test tests/Taskify.Tests/Taskify.Tests.csproj
↪  --no-build --no-restore
```

Questi passaggi compilano anche il progetto web referenziato e poi eseguono i test xUnit.

3.20.3 Workflow e hook

Spec Kit offre due meccanismi per andare oltre:

- **Hook** (in `.specify/extensions.yml`) — azioni prima/dopo i comandi. Un hook `before_specify` può creare il branch Git della feature automaticamente.
- **Workflow** — sequenze automatizzate di prompt, comandi, gate e task di shell, eseguibili anche senza interazione.

Per esempio, un workflow può incapsulare la sequenza `dotnet build` → `dotnet test` → report. È utile per ambienti in cui l'agente non è disponibile o per esecuzioni notturne; non certifica, da solo, la qualità della specifica o del piano.

3.20.4 L'automazione non sostituisce la revisione

La CI qui controlla che il codice compili e che i test passino; non verifica la qualità degli artefatti Spec Kit né che la specifica rifletta davvero ciò che serve. Per la coerenza degli artefatti si usano i controlli dell'agente, come `/speckit-analyze` e `/speckit-converge`, insieme alla revisione umana. La pipeline è una rete di sicurezza per il codice, non un sostituto del giudizio.

3.20.5 Un esempio di pipeline completa

```
name: ci

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup .NET 10
```

```
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'

      - name: Restore
        run: dotnet restore
↪  tests/Taskify.Tests/Taskify.Tests.csproj

      - name: Build
        run: dotnet build
↪  tests/Taskify.Tests/Taskify.Tests.csproj --no-restore

      - name: Test
        run: dotnet test tests/Taskify.Tests/Taskify.Tests.csproj
↪  --no-build --no-restore
```

Questa pipeline verifica il prodotto .NET senza attribuire a `specify check` o a un controllo CLI una validazione che non svolgono. La qualità del metodo e degli artefatti si rivede nel workflow Spec Kit.

3.21 Best practice e anti-pattern

Come ogni metodo, Spec Kit funziona se usato bene. Raccogliamo le pratiche che fanno la differenza e gli errori in cui è facile cadere.

3.21.1 Le best practice

Una feature, una specifica, un flusso. Non accumulare più funzionalità in una sola specifica. Una specifica focalizzata è più facile da rivisitare, pianificare e implementare.

Constitution prima di tutto. Definisci la constitution **una volta**, con cura, e falla rispettare dai gate. Una constitution vaga è inutile; una constitution chiara è il tuo "standard di qualità" codificato.

Un comando alla volta, con revisione. Non incatenare i comandi. Rivedi `spec.md` prima di pianificare, `plan.md` prima di scomporre, `tasks.md` prima di implementare. Il costo della revisione cresce con l'avanzamento.

Risolvi i `[NEEDS CLARIFICATION]`. Non lasciarli nella specifica sperando che l'agente "ci arrivi": usa `/speckit-clarify` o correggi a mano. Un requisito ambiguo è un bug in attesa.

Mantieni la specifica libera dalla tecnologia. Lo stack appartiene al piano, non alla specifica. Se nella specifica leggi "usa .NET", stai mescolando i livelli: la specifica deve dire *cosa*, il piano *come*.

Versiona tutto. Spec, piano, task e constitution sono parte del repository. La tracciabilità è il valore principale del metodo: non buttarla via con i `.gitignore`.

Itera sulla convergenza. Non accontentarti del primo `/speckit-converge` se non riporta `Converged`. Il ciclo implement→converge è ciò che garantisce la completezza.

3.21.2 Gli anti-pattern

La specifica "romanzo". Una specifica di 50 pagine di prosa è tanto inutile quanto l'assenza di specifica. Le sezioni strutturate (user story, FR, SC) esistono per questo: restare concisi e verificabili.

Saltare i gate. Ignorare `/speckit-clarify`, `/speckit-checklist` o `/speckit-analyze` "per fare prima" sposta i problemi più avanti, dove costano di più.

La constitution ipertrofica. Dieci principi generici ("il codice deve essere di qualità") non aiutano. Pochi principi **verificabili** valgono più di molti principi generici.

Fidarsi ciecamente dell'agente. L'agente produce bozze di qualità, ma le decisioni di prodotto sono tue. Rivedi, correggi, approva.

Ignorare la checklist come gate. Spuntare le caselle "tanto per" svuota il meccanismo di qualità. La checklist è di proprietà del revisore: usala con onestà.

Mescolare le feature. Modificare a mano i task di una feature per includerne un'altra rompe la tracciabilità. Se serve una nuova funzionalità, apri una nuova specifica.

3.21.3 La regola d'oro

> La specifica è tua. Spec Kit la struttura, la rende coerente e la porta fino al codice, ma **il contenuto — il cosa e il perché — è responsabilità tua**.

Se tieni fermo questo principio, Spec Kit amplificherà il tuo lavoro; se lo dimentichi, amplificherà i tuoi errori.

3.22 Il processo di bug fixing

Spec Kit non serve solo a costruire feature: include un processo dedicato alla **diagnosi e riparazione dei bug**. È un'estensione opzionale, che si installa quando serve.

Come si installa

```
specify extension add bug
```

L'estensione aggiunge tre comandi all'agente:

```
/speckit-bug-assess "Sottomettere una password vuota manda in
 ↪  crash il login." slug=login-crash
/speckit-bug-fix slug=login-crash
/speckit-bug-test slug=login-crash
```

Il flusso in tre passi

1. **Assess** — analizza il sintomo, ne individua la causa probabile e delimita l'ambito della correzione. Produce un report in `.specify/bugs/<slug>/`.
2. **Fix** — applica la correzione mirata, senza riscrivere interi moduli.
3. **Test** — verifica che il sintomo originale sia risolto e registra l'esito.

Il verdetto

Il passo finale produce un **verdetto** documentato:

Verdetto	Significato
`verified`	La correzione risolve il sintomo: successo
`partial`	Il sintomo è attenuato ma non del tutto risolto
`failed`	La correzione non risolve il sintomo

Solo `verified` conferma una correzione riuscita. Gli artefatti restano in `.specify/bugs/<slug>/` come **registrazione verificata** di ciò che è stato fatto: causa valutata, fix delimitato, verifica documentata.

Un esempio in C#

Supponiamo che la nostra Task Manager API abbia un bug: creare un task con un titolo di esattamente 201 caratteri produce un errore 500 invece di un 400. Il flusso:

```
/speckit-bug-assess "POST /tasks con titolo di 201 caratteri
 ↪  restituisce 500
invece di 400." slug=title-length-500
```

L'assessment individua la causa: la validazione controlla `Title.Length > 200` **dopo** il salvataggio, ma il database (vincolo `HasMaxLength(200)`) lancia un'eccezione prima che la validazione venga raggiunta, producendo il 500.

```
/speckit-bug-fix slug=title-length-500
```

Il fix sposta la validazione della lunghezza **prima** dell'inserimento nel database:

```
// Prima del fix: la validazione arrivava dopo il database
// Dopo il fix: la validazione avviene prima di toccare il
 ↪  DbContext
if (request.Title is { Length: > 200 })
    return Results.BadRequest(new { error = "Title must be at
     ↪  most 200 characters." });
```

```
/speckit-bug-test slug=title-length-500
```

Il test verifica che un titolo di 201 caratteri produca ora `400 Bad Request` e che il sintomo originale sia scomparso. Il verdetto finale, se tutto regge, è `verified`.

Bug fixing vs feature workflow

Il processo di bug fixing **non richiede** il workflow SDD completo: puoi diagnosticare e riparare un bug senza una specifica di feature. I due processi convivono: lo SDD per il nuovo, il bug fixing per il rotto.

3.23 Il processo di idea assessment

Il terzo processo di Spec Kit risponde a una domanda ancora più a monte: **vale la pena investire su questa idea?** Prima di scrivere specifiche o codice, l'idea assessment raccoglie evidenze per decidere.

Come si installa

```
specify extension add assess
```

L'estensione aggiunge cinque comandi:

```
/speckit-assess-intake  "Consenti agli utenti di lavorare
↪  offline." slug=offline-mode
/speckit-assess-research slug=offline-mode
/speckit-assess-define  slug=offline-mode
/speckit-assess-shape   slug=offline-mode
/speckit-assess-decide  slug=offline-mode
```

Le cinque fasi

1. **Intake** — raccoglie l'idea e ne definisce i confini.
2. **Research** — raccoglie evidenze: fattibilità tecnica, alternative esistenti, rischi.
3. **Define** — definisce il problema e il valore atteso in modo chiaro.
4. **Shape** — delinea le opzioni di soluzione e il loro costo.
5. **Decide** — produce la decisione finale.

La decisione

La decisione è espressa in tre esiti netti:

Decisione	Significato
`go`	L'idea merita investimento: si procede
`needs-clarification`	Mancano informazioni: si torna a ricercare
`kill`	L'idea non merita investimento: si ferma (documentando il perché)

Gli artefatti finiscono in `.specify/assessments/<slug>/`, con un `decision.md` finale. Se la decisione è `go`, il **riepilogo** di `decision.md` diventa l'input di `/speckit-specify`: l'idea valutata si trasforma in una specifica da costruire.

Un esempio (concettuale)

```
/speckit-assess-intake "Un sistema di cache distribuita per le
↪  nostre API."
slug=distributed-cache
```

Le fasi di ricerca e definizione potrebbero scoprire che il 90% delle API ha un carico modesto e che una cache in-process con Redis opzionale basterebbe. La decisione potrebbe essere `go` con ambito ridotto, oppure `kill` se il costo supera il beneficio. Il punto è che la decisione è **documentata ed evidenza-based**, non basata sull'intuizione del momento.

Il valore dell'assessment

In un mondo in cui generare codice è diventato economico, il **costo di perseguire l'idea sbagliata** domina. L'idea assessment sposta la riflessione ("è davvero una buona idea?") prima dell'investimento, dove è più produttiva. Fermare un'idea con una motivazione documentata è, in Spec Kit, un **risultato utile** tanto quanto un `go`.

3.24 Confronti, evoluzione e progetti esistenti

Per usare bene Spec Kit, conviene collocarlo rispetto alle pratiche che già conosci, e sapere come farlo convivere con il codice esistente.

Confronto con TDD e BDD

	TDD	BDD	SDD (Spec Kit)
Punto di partenza	Un test che fallisce	Un comportamento descritto (Given / When / Then)	Una specifica strutturata
Granularità	Classe / metodo	Feature / comportamento	Feature
Artefatto centrale	Test	Scenario eseguibile	`spec.md` + `plan.md` + `tasks.md`
Verifica della correttezza	Test verdi	Scenari verdi	Test (test-first)
Verifica della completezza	Implicita	Implicita	`/speckit-converge`

I tre approcci **non si escludono**: Spec Kit, anzi, *impone* il test-first nei suoi task, e usa il formato Given/When/Then negli scenari di accettazione. SDD è TDD/BDD "scalato" al livello della feature, con in più una **specifica esplicita** e una **verifica di completezza**.

Confronto con i design doc tradizionali

Un design doc tradizionale e un `plan.md` di Spec Kit coprono terreni simili, ma differiscono in tre aspetti:

1. **Struttura vincolata.** Il `plan.md` segue un template rigido (Technical Context, Constitution Check, Project Structure), mentre il design doc è in prosa libera. La struttura costante rende i piani confrontabili tra feature.
2. **Tracciabilità.** Il piano Spec Kit è collegato a una specifica (`spec.md`) e genera task (`tasks.md`): è un anello di una catena, non un documento isolato.
3. **Verifica.** Il piano passa da un **gate** (Constitution Check) e da una verifica di coerenza (`/speckit-analyze`), cosa che i design doc tradizionali non hanno.

Far evolvere una specifica esistente

Il software evolve, e così le specifiche. Spec Kit prevede esplicitamente questo caso: quando una feature cambia (nuovi requisiti, chiarimenti, correzioni), si **aggiorna la specifica** e si ripropaga il cambiamento lungo la catena:

1. modifica `spec.md` (a mano o con l'agente);
2. riallinea `plan.md` se il cambiamento tocca il *come*;
3. rigenera o aggiorna `tasks.md`;
4. riesegui `/speckit-analyze` per verificare la coerenza;
5. implementa e converge.

La regola è: **non modificare mai il codice scavalcando la specifica**. Se il codice cambia, la specifica deve cambiare prima (o insieme): è ciò che mantiene la tracciabilità viva nel tempo.

Adottare Spec Kit su un progetto esistente

Spec Kit non richiede un progetto nuovo. Su un repository esistente:

1. esegui `specify init . --integration <agente>` per aggiungere la struttura;
2. scrivi una **constitution** che rifletta gli standard già in uso (non imporre principi estranei);
3. parti dalle **nuove feature**: usa il flusso SDD completo per ciò che arriva;
4. per il codice esistente, procedi per gradi: puoi scrivere specifiche "a posteriori" per i moduli critici, o usare il processo di bug fixing per le riparazioni.

L'adozione incrementale è esplicitamente incoraggiata: "inizia con il cambiamento più piccolo che soddisfa le tue esigenze; non devi sostituire l'intero processo".

Quando (non) usare Spec Kit

Spec Kit brilla per feature **non banali** e per team in cui la revisione delle specifiche ha valore. Per una correzione di una riga o un prototipo usa-e-getta, il flusso completo sarebbe un overhead: in quei casi, il processo di bug fixing o un semplice dialogo con l'agente sono più adatti. La maturità sta nel saper scegliere, non nell'applicare sempre lo stesso processo.

3.25 Risoluzione dei problemi comuni

Raccogliamo i problemi più frequenti nell'uso di Spec Kit, con causa e soluzione.

Il comando `/speckit-*` non viene trovato dall'agente

Causa. L'integrazione non è stata configurata: `specify init` è stato eseguito senza `--integration`, oppure l'agente non ha ricaricato le skill dopo l'init.

Soluzione. Esegui `specify init --integration <agente>` (o riapri il progetto/riavvia l'agente perché rilevi le nuove skill). Verifica che i comandi compaiano nell'elenco delle skill dell'agente.

`specify` non è riconosciuto come comando

Causa. Il CLI non è installato o il percorso di `uv` non è nel `PATH`.

Soluzione. Reinstalla con `uv tool install specify-cli` e verifica con `specify --version`; se necessario, aggiungi la directory dei tool di `uv` al `PATH`.

Il comando lavora sulla feature sbagliata

Causa. I comandi risolvono la feature da `.specify/feature.json` (o da `SPECIFY_FEATURE_DIRECTORY`), **non** dal branch Git. Se hai cambiato branch senza aggiornare lo stato, il comando continua a puntare alla feature precedente.

Soluzione. Controlla `.specify/feature.json`: deve contenere il percorso della feature giusta. Aggiornalo (o imposta `SPECIFY_FEATURE_DIRECTORY`) prima di eseguire i comandi.

`/speckit-implement` si rifiuta di partire

Causa. La checklist `checklists/requirements.md` ha voci non spuntate: è il **gate** che funziona come previsto.

Soluzione. Rivedi le voci non spuntate: o le spunti (in veste di revisore) o correggi la specifica e riesegui il controllo.

`/speckit-analyze` segnala conflitti

Causa. Gli artefatti si sono disallineati (per esempio hai modificato la specifica a mano dopo aver generato i task).

Soluzione. Correggi gli artefatti alla fonte (specifica, piano o task) per riallinearli, poi riesegui `/speckit-analyze`. Non ignorare le segnalazioni e non procedere con l'implementazione.

`/speckit-converge` aggiunge task all'infinito

Causa. Se a ogni iterazione converge aggiunge nuovi task senza arrivare a `Converged`, di solito c'è un disallineamento sistematico (per esempio un requisito ambiguo che l'implementazione interpreta in modo diverso dalla specifica).

Soluzione. Fermati e rileggi la specifica: probabilmente va **chiarita** (torna a `/speckit-clarify` o modificala a mano). Un requisito chiaro produce un'implementazione convergente.

Il comando ha una sintassi diversa da quella dell'esempio

Causa. Le skill e i comandi esposti dipendono dall'agente e dall'integrazione usata per generarle. La differenza tra punto e trattino, da sola, non identifica una versione obsoleta.

Soluzione. Controlla l'elenco dei comandi o i file di skill generati nel progetto e usa la sintassi prevista dalla tua integrazione. Gli esempi di questo capitolo adottano la forma con trattino.

`specify check` segnala prerequisiti mancanti

Causa. Uno strumento richiesto dal CLI non è installato o non è disponibile nel `PATH`, oppure l'ambiente usa una configurazione diversa da quella locale.

Soluzione. Esegui `specify check` nello stesso ambiente per identificare il prerequisito mancante. Il comando non convalida gli artefatti; usa la revisione del workflow, `/speckit-analyze` e `/speckit-converge` per controllarli. `specify self check`, se disponibile, diagnostica il CLI e non sostituisce quei controlli.

Metodo di diagnosi universale

Quando qualcosa non va, riduci il problema: verifica la **configurazione** (`.specify/feature.json`, integrazione), poi gli **artefatti** (specifica, piano, task allineati?), poi il **codice** (build e test). Quasi sempre l'errore è a monte di dove sembra.

3.26 Riepilogo, glossario ed esercizi

Siamo alla fine del capitolo. Ripercorriamo il cammino e consolidiamo i concetti.

3.26.1 Riepilogo

In questo capitolo abbiamo:

- introdotto lo **spec-driven development** e il suo principio guida: la specifica come fonte di verità;
- presentato **GitHub Spec Kit** come toolkit open source per agenti di codifica, con i suoi tre processi (SDD, bug fixing, idea assessment);

- installato il **CLI `specify`** e inizializzato un progetto con integrazione Copilot;
- esplorato la **struttura** del progetto (`.specify/`, `specs/`, `feature.json`, constitution, template);
- percorso il **workflow completo**: constitution, specify, clarify, plan, checklist, tasks, analyze, implement, polish e converge;
- studiato i **template ufficiali** (spec, plan, tasks, checklist, constitution) e i loro meccanismi (priorità, Given/When/Then, `[NEEDS CLARIFICATION]`, Constitution Check, test-first);
- applicato l'intero flusso a una feature **C#**: una Task Manager API con Minimal API, EF Core e test xUnit, dalla specifica al codice funzionante;
- visto l'integrazione con **VS Code e Copilot** e l'automazione in **CI/CD**;
- raccolto **best practice** e soluzioni ai problemi comuni.

3.26.2 Glossario

Termine	Definizione
SDD	Specification-Driven Development: sviluppo guidato dalla specifica
Spec Kit	Toolkit open source di GitHub per applicare lo SDD con gli agenti AI
Constitution	Principi stabili del progetto, emendabili secondo la governance (`.specify/memory/constitution.md`)
Spec	La specifica della feature (`spec.md`)
Plan	Il piano tecnico (`plan.md`) con modello dati e contratti
Tasks	La lista di task eseguibili, ordinati per dipendenza (`tasks.md`)
Checklist	Checklist dei requisiti (`checklists/requirements.md`) e checklist aggiuntive per ambito
Converge	Verifica che il codice corrisponda a spec, plan e task
Feature attiva	La feature su cui lavorano i comandi (in `.specify/feature.json`)
Hook	Azioni eseguite prima/dopo i comandi (`.specify/extensions.yml`)

Termine	Definizione
Preset/Workflow/Bundle	Meccanismi per adattare e distribuire processi

3.26.3 Esercizi proposti

1. **La tua constitution.** Scrivi (o genera con `/speckit-constitution`) una constitution per un progetto .NET di tua scelta: 4 principi verificabili, una sezione di sicurezza e una di governance.
2. **Una specifica.** Genera con `/speckit-specify` la specifica di una feature che conosci bene (per esempio un endpoint di autenticazione). Poi verifica a mano che rispetti i criteri della checklist (priorità, Given/When/Then, FR numerati, SC misurabili).
3. **Chiarimento.** Individua tre `[NEEDS CLARIFICATION]` in una specifica esistente e risolvili con `/speckit-clarify`.
4. **Piano e task.** A partire dalla specifica dell'esercizio 2, genera piano e task, e verifica che ogni user story abbia i suoi task e i suoi test.
5. **Implementazione.** Estendi la Task Manager API con una data di completamento registrata (oltre allo stato booleano): modifica specifica, piano, task e codice, poi esegui `/speckit-converge`.
6. **CI.** Aggiungi un workflow GitHub Actions che compili il progetto ed esegua `dotnet test tests/Taskify.Tests/Taskify.Tests.csproj`; usa `specify check` solo per diagnosticare i prerequisiti del CLI, non come gate degli artefatti.
7. **Bug fixing.** Installa l'estensione `bug` e usala su un bug reale di un tuo progetto: osserva come il verdetto finale (`verified/partial/failed`) documenta l'esito.

3.26.4 Il filo conduttore

Tre capitoli, tre tasselli. Gli **strumenti** (i server MCP) permettono all'agente di agire; i **dati** (EF Core) gli danno continuità; il **metodo** (Spec Kit) lo guida con un processo. Solo insieme trasformano un modello linguistico in un compagno di sviluppo affidabile.